

BOT AUDIT CODE REVIEW

Generated: 2026-05-30 11:55:27

Focus: Core Trading Logic & Execution

Source: C:\Users\pc\Desktop\projects\Quad-desk\bot

Core Components:

main.py - Bot Entry Point & Main Loop

quant_engine.py - Quantitative Trading Engine

executor.py - Trade Execution Logic

signal_config.py - Signal Configuration & Parameters

ulis_engine.py - ULIS Strategy Engine

data_feed.py - Market Data Handler

derivatives_context.py - Derivatives Context Manager

notifier.py - Alert & Notification System

heartbeat.py - Heartbeat & Monitoring

backtest_hybrid.py - Backtesting Engine

Note: This audit PDF contains core bot logic with optimized formatting for readability. Redundant comments and docstrings have been removed.

TABLE OF CONTENTS

1. main.py - Bot Entry Point & Main Loop
2. quant_engine.py - Quantitative Trading Engine
3. executor.py - Trade Execution Logic
4. signal_config.py - Signal Configuration & Parameters
5. ulis_engine.py - ULIS Strategy Engine
6. data_feed.py - Market Data Handler
7. derivatives_context.py - Derivatives Context Manager
8. notifier.py - Alert & Notification System
9. heartbeat.py - Heartbeat & Monitoring
10. backtest_hybrid.py - Backtesting Engine

■ main.py

Bot Entry Point & Main Loop

```
import asyncio
import logging
import os
import re
import signal
import time
import numpy as np
from collections import deque
from datetime import date
from typing import Dict, Any, Optional, Tuple, List

from dotenv import load_dotenv
load_dotenv()

from bot.data_feed import BinanceDataFeed
from bot.quant_engine import QuantEngine
from bot.executor import TradingExecutor
from bot.ulis_engine import compute_ulis_verdict
from bot.derivatives_context import DerivativesContext
from bot import heartbeat
from bot.signal_config import (
    REGIME_PARAMS, POST_TRADE_COOLDOWN_S, COLD_START_TRADE_COUNT,
    COLD_START_CONFIDENCE_DISCOUNT,
    MAX_RISK_PCT as CFG_MAX_RISK_PCT,
    MAX_DAILY_LOSS_PCT as CFG_MAX_DAILY_LOSS_PCT,
    MAX_DRAWDOWN_PCT as CFG_MAX_DRAWDOWN_PCT,
    MIN_BAYESIAN as CFG_MIN_BAYESIAN
)

# #####
# Logging
# #####
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s | %(levelname)-8s | %(name)s | %(message)s',
    datefmt='%H:%M:%S',
)
logger = logging.getLogger("bot.main")

try:
    from bot.heartbeat import FirestoreLogHandler
    logging.getLogger().addHandler(FirestoreLogHandler())
except Exception as e:
    logger.warning(f"[Main] Could not attach FirestoreLogHandler: {e}")

# #####
# Config from environment
# #####
EXCHANGE = os.environ.get("BOT_EXCHANGE", "binanceusdm").lower()
SYMBOL = os.environ.get("BOT_SYMBOL", "BTC-USDC" if EXCHANGE == "coinbase" else "BTC/USDT")
TESTNET = os.environ.get("BOT_TESTNET", "false").lower() != "false"
MAX_RISK_PCT = float(os.environ.get("BOT_MAX_RISK_PCT", str(CFG_MAX_RISK_PCT)))
MAX_DAILY_LOSS_PCT = float(os.environ.get("BOT_MAX_DAILY_LOSS_PCT", str(CFG_MAX_DAILY_LOSS_PCT)))
ANALYSIS_INTERVAL = int(os.environ.get("BOT_ANALYSIS_INTERVAL", "15"))
CANDLE_INTERVAL = os.environ.get("BOT_CANDLE_INTERVAL", "15m")
MIN_CONFIDENCE = float(os.environ.get("BOT_MIN_CONFIDENCE", str(CFG_MIN_BAYESIAN)))
ACCOUNT_SIZE = float(os.environ.get("BOT_ACCOUNT_SIZE", "100.0"))
LEVERAGE = int(os.environ.get("BOT_LEVERAGE", "3")) # futures leverage (3x = efficient margin on Binance)
ULIS_GATE_ENABLED = os.environ.get("BOT_ULIS_GATE", "true").lower() != "false"

# Fee rates: Coinbase Spot 1.2% | Binance Spot 0.1% | Binance USDM Futures 0.04%
_EXCHANGE_FEE_RATE = 0.012 if EXCHANGE == "coinbase" else (0.0004 if EXCHANGE == "binanceusdm" else 0.001)

# Panic mode: trigger if price drops this % within PANIC_LOOKBACK candles
# e.g. 3.0 = 3% crash within 5 candles → engage panic mode
PANIC_DROP_PCT = float(os.environ.get("BOT_PANIC_DROP_PCT", "3.0"))
PANIC_LOOKBACK = int(os.environ.get("BOT_PANIC_LOOKBACK", "5")) # candles
PANIC_LOCK_SECONDS = int(os.environ.get("BOT_PANIC_LOCK_SECONDS", "300")) # 5 min default
MAX_DRAWDOWN_PCT = float(os.environ.get("BOT_MAX_DRAWDOWN_PCT", str(CFG_MAX_DRAWDOWN_PCT)))

# Coinbase credentials
CB_KEY_NAME = os.environ.get("COINBASE_API_KEY_NAME", "")
CB_PRIVATE_KEY = os.environ.get("COINBASE_PRIVATE_KEY", "")

# Binance credentials
BINANCE_API_KEY = os.environ.get("BINANCE_API_KEY", "")
BINANCE_API_SECRET = os.environ.get("BINANCE_API_SECRET", "")
BINANCE_ED25519_PRIVKEY = os.environ.get("BINANCE_ED25519_PRIVATE_KEY", "")
# Normalise escaped newlines from .env (stored as literal \n)
if BINANCE_ED25519_PRIVKEY:
    BINANCE_ED25519_PRIVKEY = BINANCE_ED25519_PRIVKEY.replace("\\n", "\n").strip()

# Telegram credentials
TG_BOT_TOKEN = os.environ.get("TELEGRAM_BOT_TOKEN", "")
TG_CHAT_ID = os.environ.get("TELEGRAM_CHAT_ID", "")
# Derive dry-run: explicit override or no credentials at all
```



```

f" zscore_warmup      : {g.get('zscore_warmup', 0)}\n"
f" post_trade_cooldown : {g.get('post_trade_cooldown', 0)}\n"
f" cascade_cooldown   : {g.get('cascade_cooldown', 0)}\n"
f" regime_no_edge      : {g.get('regime_no_edge', 0)}\n"
f" htf_counter_trend   : {g.get('htf_counter_trend', 0)}\n"
f" funding_blocks      : {g.get('funding_blocks_long', 0) + g.get('funding_blocks_short', 0)}\n"
f" ulis_veto           : {g.get('ulis_veto', 0)}\n"
f" ulis_alignment_fail : {g.get('ulis_alignment_fail', 0)}\n"
f" low_confidence       : {g.get('confidence_below_threshold', 0)}\n"
f" candle_gate_expired : {g.get('candle_gate_expired', 0)}\n"
f" micro_confirms_fail : {g.get('micro_confirms_failed', 0)}\n"
f" sweep_confirms_fail : {g.get('sweep_confirms_failed', 0)}\n"
f" fee_geometry         : {g.get('fee_geometry', 0)}\n"
f" signal_none          : {g.get('signal_none', 0)}\n"
f" #####\n"
f" total_rejections     : {total}\n"
f" total_passed         : {passed}\n"
f" pass_rate            : {passed/max(1, passed+total):.1%}\n"
f" last_confidence      : {confidence:.2%}"
)
GATE_STATS_LAST_LOG = now

# #####
# ■■■ STAGE 1 – FEATURE ENGINE HELPERS ■■■#####
# #####

def _parse_walls(all_walls_str: Optional[str]) -> Tuple[List[float], List[float]]:
    if not all_walls_str:
        return [], []
    buy_walls, sell_walls = [], []
    for part in all_walls_str.split(";"):
        m = re.match(r"\s*(BUY|SELL)@([\d.]+)", part.strip())
        if m:
            (buy_walls if m.group(1) == "BUY" else sell_walls).append(float(m.group(2)))
    buy_walls.sort(reverse=True)
    sell_walls.sort()
    return buy_walls, sell_walls

# #####
# ■■■ STAGE 2 – MARKET REGIME DETECTION ■■■#####
# #####

def _htf_trend(candle_history: list) -> str:
    if len(candle_history) < 16:
        return "NEUTRAL"
    # The open of the 16th candle ago represents the start of the 4H window
    window_open = float(candle_history[-16]["open"])
    htf_close = float(candle_history[-1]["close"])
    if window_open <= 0:
        return "NEUTRAL"
    change_pct = (htf_close - window_open) / window_open
    # HIGH-3 FIX: 0.2% was too tight (BTC moves that in minutes).
    # 0.5% over 4H is a meaningful, non-noise directional move.
    if change_pct > 0.005: # +0.5% over 4H = clear uptrend
        return "BULL"
    if change_pct < -0.005: # -0.5% over 4H = clear downtrend
        return "BEAR"
    return "NEUTRAL"

def _vpoc_confidence_boost(price: float, vpoc: Optional[float], direction: str) -> float:
    if vpoc is None or price <= 0:
        return 0.0
    dist_pct = abs(price - vpoc) / price # distance as fraction of price
    is_long = direction in ("BUY", "MEAN_REVERSAL_LONG")

    aligned = (is_long and price <= vpoc * 1.002) or (not is_long and price >= vpoc * 0.998)
    if not aligned:
        return 0.0

    # Graduated boost: 0.06 when at VPOC, falling to 0 at 0.3% away
    if dist_pct <= 0.003:
        return round(0.06 * (1.0 - dist_pct / 0.003), 4)
    return 0.0

async def _derivatives_gate(direction: str, deriv_context: dict) -> tuple[bool, str]:
    is_long = direction in ("BUY", "MEAN_REVERSAL_LONG")

    oi = deriv_context.get("open_interest", {})
    tt = deriv_context.get("top_traders", {})
    opts = deriv_context.get("options", {})
    flow = deriv_context.get("taker_flow", {})

    crowd = tt.get("crowd_signal", "NEUTRAL")
    opts_signal = opts.get("options_signal", "NEUTRAL")
    oi_collapsing = oi.get("oi_collapsing", False)
    taker_imbalance = flow.get("taker_imbalance", 0.0)
    veto_reasons = []

```

```
# VETO 1: Top traders crowded in SAME direction as our trade = no edge
# 70%+ of whales already positioned same as us = crowded trade, squeeze risk
if is_long and crowd == "CROWDED_LONG":
    veto_reasons.append("Top traders 70%+ LONG - crowded, no squeeze potential")
if not is_long and crowd == "CROWDED_SHORT":
    veto_reasons.append("Top traders 70%+ SHORT - crowded, no squeeze potential")

# VETO 2: Options market in FEAR while we want to buy
# Institutions paying for downside puts = they expect lower prices
if is_long and opts_signal == "FEAR":
    veto_reasons.append("Options FEAR - institutions hedging downside aggressively")

# VETO 3: OI collapsing = deleveraging, not new trend forming
if oi_collapsing:
    oi_mom = oi.get("oi_momentum_1h", 0.0)
    veto_reasons.append(f"OI collapsing {oi_mom:.1f}% in 1h - deleveraging, no new trend")

if veto_reasons:
    return False, " | ".join(veto_reasons)

# BOOST signals (logged but don't force entry)
boosts = []
if is_long and crowd == "CROWDED_SHORT":
    boosts.append("SHORT SQUEEZE SETUP - whales crowded short")
if not is_long and crowd == "CROWDED_LONG":
    boosts.append("LONG SQUEEZE SETUP - whales crowded long")
if abs(taker_imbalance) > 0.3:
    direction_match = (is_long and taker_imbalance > 0) or (not is_long and taker_imbalance < 0)
    if direction_match:
        boosts.append(f"Taker flow confirms: imbalance={taker_imbalance:.2f}")

return True, " | ".join(boosts) if boosts else "No institutional conflict"
```

```
# HMM REGIME CLASSIFIER (3-state, pure numpy)
```

```
class _HMMRegimeClassifier:
```

```
    # --- Emission means ( $\mu$ ) per state x feature -----
    # Features: [atr_pct, |z_score|, tape_binary, atr_pct_rank]
    # f0 = atr_pct      (0 - 0.02) raw ATR as fraction of price
    # f1 = abs(z_score) (0 - 4)
    # f2 = tape_binary  (0 = NORMAL, 1 = SCREAMING)
    # f3 = atr_pct_rank (0 - 1) percentile rank in 30-day window (P2)
    _MU = np.array([
        [0.003, 0.8, 0.05, 0.20], # RANGE
        [0.007, 1.5, 0.30, 0.55], # TREND - retained at 0.007 per CPO
        [0.014, 2.5, 0.65, 0.85], # VOLATILE
    ], dtype=float)

    _SIGMA = np.array([
        [0.0010, 0.40, 0.10, 0.10], # RANGE - COMPRESSED (was 0.0015, 0.6, 0.15, 0.15)
        [0.0030, 0.80, 0.25, 0.20], # TREND
        [0.0050, 1.00, 0.30, 0.15], # VOLATILE
    ], dtype=float)

    # --- Transition matrix (rows = from-state, cols = to-state) -----
    # 80% persistence, 10% to each neighbour state
    _A = np.array([
        [0.82, 0.12, 0.06],
        [0.10, 0.80, 0.10],
        [0.06, 0.12, 0.82],
    ], dtype=float)

    # --- Initial state distribution -----
    _PI = np.array([0.50, 0.35, 0.15], dtype=float)

    # State labels (index → regime string)
    _LABELS = ["RANGE", "TREND", "NEUTRAL"] # VOLATILE maps to NEUTRAL

    # Confidence & hysteresis thresholds
    MIN_CONFIDENCE = 0.60
    HYSTERESIS_CANDLES = 2 # FIXED: 30-min lag (was 3 = 45-min lag)
    FAST_TRACK_CONFIDENCE = 0.88 # NEW: 1-candle commit at very high confidence

    def __init__(self, window: int = 60, update_every: int = 50):
        self.window = window
        self.update_n = update_every
        self._obs_buf = deque(maxlen=200) # circular feature history
        self._cycle = 0 # count calls since last param update
        # Mutable copies so online-update can adjust them
        self._mu = self._MU.copy()
        self._sigma = self._SIGMA.copy()
        # Hysteresis state
        self._committed_regime = "RANGE" # currently committed regime
        self._candidate_regime = "RANGE" # regime the HMM is suggesting
        self._candidate_streak = 0 # consecutive candles suggesting candidate

        self.persist_path = os.environ.get("HMM_STATE_PATH", "/tmp/quadrant_hmm_state.json")
```

```

self._load_state()

def _save_state(self):
    import threading
    import json
    import time
    data = {
        "mu": self._mu.tolist(),
        "sigma": self._sigma.tolist(),
        "saved_at": time.time(),
    }

def _write_firestore(payload: dict) -> None:
    try:
        import json
        fs_payload = payload.copy()
        fs_payload["mu"] = json.dumps(fs_payload.get("mu", []))
        fs_payload["sigma"] = json.dumps(fs_payload.get("sigma", []))
        from bot.heartbeat import get_db
        db = get_db()
        if db is not None:
            db.collection("botState").document("hmmEngine").set(fs_payload)
            logger.debug("[HMM] State saved to Firestore █")
    except Exception as e:
        logger.warning(f"[HMM] Firestore state save failed: {e}")

    threading.Thread(
        target=_write_firestore,
        args=(dict(data),),
        daemon=True,
        name="HMMEngine-FSWrite",
    ).start()

    try:
        with open(self._persist_path, "w") as f:
            json.dump(data, f)
    except Exception as e:
        pass

def _load_state(self):
    import json
    import time
    try:
        from bot.heartbeat import get_db
        db = get_db()
        if db is not None:
            doc = db.collection("botState").document("hmmEngine").get()
            if doc.exists:
                data = doc.to_dict()
                mu_val = data.get("mu")
                sig_val = data.get("sigma")
                if isinstance(mu_val, str): mu_val = json.loads(mu_val)
                if isinstance(sig_val, str): sig_val = json.loads(sig_val)
                self._mu = np.array(mu_val if mu_val is not None else self._MU.tolist())
                self._sigma = np.array(sig_val if sig_val is not None else self._SIGMA.tolist())
                logger.info("[HMM] █ State loaded from Firestore")
                return
    except Exception as e:
        logger.warning(f"[HMM] Firestore state load failed: {e} – falling back to /tmp/")

    try:
        import os
        if os.path.exists(self._persist_path):
            with open(self._persist_path) as f:
                data = json.load(f)
                self._mu = np.array(data.get("mu", self._MU.tolist()))
                self._sigma = np.array(data.get("sigma", self._SIGMA.tolist()))
                logger.info("[HMM] █ State loaded from /tmp/")
                return
    except Exception as e:
        pass

# -----
def _gaussian_log_prob(self, obs: np.ndarray) -> np.ndarray:
    log_probs = np.zeros(3)
    for s in range(3):
        diff = obs - self._mu[s]
        log_p = -0.5 * np.sum((diff / np.maximum(self._sigma[s], 1e-9)) ** 2)
        log_p -= np.sum(np.log(np.maximum(self._sigma[s], 1e-9)))
        log_probs[s] = log_p
    return log_probs

# -----
@staticmethod
def _logsumexp(a: np.ndarray) -> float:
    a_max = np.max(a)
    if not np.isfinite(a_max):
        return a_max

```

```

        return a_max + np.log(np.sum(np.exp(a - a_max)))
# -----
def _forward(self, obs_seq: np.ndarray) -> np.ndarray:
    T = len(obs_seq)
    n_s = 3
    log_A = np.log(np.maximum(self._A, 1e-300))
    log_pi = np.log(np.maximum(self._PI, 1e-300))

    # log_alpha[t, s] = log P(o_1...o_t, state_t = s)
    log_alpha = np.full((T, n_s), -np.inf)
    log_alpha[0] = log_pi + self._gaussian_log_prob(obs_seq[0])

    for t in range(1, T):
        log_emit = self._gaussian_log_prob(obs_seq[t])
        for j in range(n_s):
            # Sum over all possible previous states (marginalise)
            log_alpha[t, j] = self._logsumexp(
                log_alpha[t - 1] + log_A[:, j]
            ) + log_emit[j]

    # Normalise to get posterior at time T
    log_evidence = self._logsumexp(log_alpha[-1])
    log_posterior = log_alpha[-1] - log_evidence
    posterior = np.exp(log_posterior)

    # Safety: ensure sums to 1.0 (numerical precision)
    posterior = np.nan_to_num(posterior, nan=0.0, posinf=0.0, neginf=0.0)
    posterior = np.maximum(posterior, 0.0)
    total = posterior.sum()
    if total > 0:
        posterior /= total
    else:
        posterior = np.array([0.50, 0.35, 0.15]) # fallback to prior

    return posterior
# -----
def _viterbi(self, obs_seq: np.ndarray) -> np.ndarray:
    T, _ = obs_seq.shape
    n_s = 3
    log_A = np.log(np.maximum(self._A, 1e-300))
    log_pi = np.log(np.maximum(self._PI, 1e-300))

    delta = np.full((T, n_s), -np.inf)
    psi = np.zeros((T, n_s), dtype=int)

    delta[0] = log_pi + self._gaussian_log_prob(obs_seq[0])

    for t in range(1, T):
        log_emit = self._gaussian_log_prob(obs_seq[t])
        for j in range(n_s):
            trans = delta[t - 1] + log_A[:, j]
            psi[t, j] = int(np.argmax(trans))
            delta[t, j] = np.max(trans) + log_emit[j]

    # Back-track
    states = np.zeros(T, dtype=int)
    states[-1] = int(np.argmax(delta[-1]))
    for t in range(T - 2, -1, -1):
        states[t] = psi[t + 1, states[t + 1]]
    return states
# -----
def _online_update(self):
    if len(self._obs_buf) < 20:
        return
    obs = np.array(list(self._obs_buf)[-self._window:], dtype=float)
    states = self._viterbi(obs)
    for s in range(3):
        mask = states == s
        if mask.sum() >= 3:
            self._mu[s] = obs[mask].mean(axis=0)
            self._sigma[s] = np.maximum(obs[mask].std(axis=0), 1e-4)

    self._save_state()
# -----
def classify(self, atr_pct: float, z_score: float, tape: str,
            atr_pct_rank: float = 0.5) -> dict:

    obs = np.array([
        float(np.clip(atr_pct, 0.0, 0.03)),
        float(np.clip(abs(z_score), 0.0, 4.0)),
        1.0 if tape == "SCREAMING" else 0.0,
        float(np.clip(atr_pct_rank, 0.0, 1.0)), # P2: 4th feature
    ], dtype=float)

```



```

self._obs_buf.append(obs)
self._cycle += 1

if self._cycle % self._update_n == 0:
    self._online_update()

# Need at least 3 observations for a meaningful forward pass
n_obs = min(len(self._obs_buf), self._window)
seq = np.array(list(self._obs_buf)[-n_obs:], dtype=float)

if len(seq) < 3:
    # Fall back to simple thresholds while warming up
    if atr_pct > 0.01 and tape == "SCREAMING":
        raw = "TREND"
    elif abs(z_score) > 2.5:
        raw = "NEUTRAL"
    else:
        raw = "RANGE"
    return {
        "regime": raw, "confidence": 0.50,
        "p_range": 0.33, "p_trend": 0.33, "p_volatile": 0.33,
        "raw_regime": raw,
    }

# ■■ Forward algorithm: posterior probability vector ■■■■■■■■■■■■
posterior = self._forward(seq)
best_state = int(np.argmax(posterior))
raw_label = self._LABELS[best_state]
raw_conf = float(posterior[best_state])

# ■■ Hysteresis: 3-candle debounce on regime transitions ■■■■■■
# Prevents flickering at regime boundaries (e.g. TREND→RANGE→TREND
# on consecutive cycles when the posterior is near 50/50).
if raw_label == self._candidate_regime:
    self._candidate_streak += 1
else:
    self._candidate_regime = raw_label
    self._candidate_streak = 1

# Commit the new regime only if it's been consistent for N candles
# AND meets the confidence threshold
_fast = (raw_conf >= self.FAST_TRACK_CONFIDENCE
        and self._candidate_regime != self._committed_regime)
_normal = (self._candidate_streak >= self.HYSTERESIS_CANDLES
          and raw_conf >= self.MIN_CONFIDENCE
          and self._candidate_regime != self._committed_regime)

if _fast or _normal:
    old = self._committed_regime
    self._committed_regime = self._candidate_regime
    logger.info(
        f"[HMM] Regime TRANSITION ({'FAST' if _fast else 'normal'}): "
        f"{old} → {self._committed_regime} "
        f"(conf={raw_conf:.1%}, streak={self._candidate_streak})"
    )

# The committed regime's confidence is its actual posterior probability
committed_idx = self._LABELS.index(self._committed_regime)
committed_conf = float(posterior[committed_idx])

logger.debug(
    f"[HMM] raw={raw_label}({raw_conf:.0%}) committed={self._committed_regime}"
    f"({committed_conf:.0%}) streak={self._candidate_streak} | "
    f"P=[R:{posterior[0]:.0%} T:{posterior[1]:.0%} V:{posterior[2]:.0%}]"
)

return {
    "regime": self._committed_regime,
    "confidence": committed_conf,
    "p_range": float(posterior[0]),
    "p_trend": float(posterior[1]),
    "p_volatile": float(posterior[2]),
    "raw_regime": raw_label,
}

# Module-level singleton – persists observations across cycles
_hmm_classifier = _HMMRegimeClassifier(window=60, update_every=200)

# Derivatives context for institutional signals
_derivatives_ctx = DerivativesContext(symbol=FEED_SYMBOL)

def _detect_regime(
    metrics: Dict[str, Any],
    buy_walls: List[float],
    sell_walls: List[float],
    quant, # PHASE-1.3: Reference to QuantEngine for stateful tracking
    sweep: Optional[str] = None,
) -> str:
    price = metrics["price"]
    z = metrics["zScore"]

```

[illegible]


```

cvd_bear = cvd < 0
tape_bear = "SELL" in dominant
micro_confirms_bear = sum([ofi_bear, cvd_bear, tape_bear])

if score >= 1.5:
    # Intended LONG
    ofi_cvd_aligned = (ofi > 0.15 and cvd > 0)
    if micro_confirms_bull < 2 and score < 3.0 and not ofi_cvd_aligned:
        BOT_STATS["gate_stats"]["micro_confirms_failed"] = BOT_STATS["gate_stats"].get("micro_confirms_failed", 0) + 1
        logger.info(f"[TrendStrategy] score={score:.2f} rejected LONG (micro_confirms_bull={micro_confirms_bull}/3, OFI+CVD)
        return None
    return "BUY"

if score <= -1.5:
    # Intended SHORT
    ofi_cvd_aligned = (ofi < -0.15 and cvd < 0)
    if micro_confirms_bear < 2 and score > -3.0 and not ofi_cvd_aligned:
        BOT_STATS["gate_stats"]["micro_confirms_failed"] = BOT_STATS["gate_stats"].get("micro_confirms_failed", 0) + 1
        logger.info(f"[TrendStrategy] score={score:.2f} rejected SHORT (micro_confirms_bear={micro_confirms_bear}/3, OFI+CVD)
        return None
    return "SELL"

return None

def _strategy_mean_reversion(metrics: Dict[str, Any], z_threshold: float = 1.5) -> Optional[str]:
    z = metrics.get("zScore", 0.0)
    rsi = metrics.get("rsi", 50.0)
    atr_rank = metrics.get("atr_pct_rank", 0.5)
    if atr_rank < 0.5:
        rsi_long_gate = 55.0
        rsi_short_gate = 45.0
    else:
        rsi_long_gate = 42.0
        rsi_short_gate = 58.0
    if z >= z_threshold and rsi > rsi_short_gate:
        return "MEAN_REVERSAL_SHORT"
    if z <= -z_threshold and rsi < rsi_long_gate:
        return "MEAN_REVERSAL_LONG"
    return None

def _strategy_liquidity_sweep(sweep: str, metrics: Dict[str, Any]) -> Optional[str]:
    ofi = metrics["ofi"]
    cvd = metrics["cvd"]
    dominant = metrics["tapeDominant"]
    z = metrics.get("zScore", 0.0)
    rsi = metrics.get("rsi", 50.0)

    # P1-1 FIX: Z+RSI overbought/oversold gate on sweep entries.
    # The Apr-23 audit caught Z=+2.67, RSI=70 entering a BELOW_LOWS BUY sweep –
    # price was already statistically overbought; the sweep was a continuation,
    # not a reversal. Block these momentum-chase entries at the strategy level.
    if sweep == "BELOW_LOWS" and z > 2.0 and rsi > 65:
        logger.info(
            f"[SweepGate] BUY sweep BLOCKED – Z={z:.2f} RSI={rsi:.1f} "
            f"(price already overbought, sweep likely continuation not reversal)"
        )
        return None
    if sweep == "ABOVE_HIGHS" and z < -2.0 and rsi < 35:
        logger.info(
            f"[SweepGate] SELL sweep BLOCKED – Z={z:.2f} RSI={rsi:.1f} "
            f"(price already oversold, sweep likely continuation not reversal)"
        )
        return None

    # PHASE-2.1 FIX: Lower confirms threshold 3/3 → 2/3 for sweep entries.
    # in live markets. 2/3 requires OFI+CVD+tape to agree, still enforcing quality.
    if sweep == "ABOVE_HIGHS":
        confirms = sum([
            ofi < -0.15,
            cvd < 0,
            "SELL" in dominant,
        ])
        if confirms >= 2:
            logger.info(f"[SweepStrat] SELL after ABOVE_HIGHS (confirms={confirms}/3)")
            return "SELL"

    if sweep == "BELOW_LOWS":
        confirms = sum([
            ofi > 0.15,
            cvd > 0,
            "BUY" in dominant,
        ])
        if confirms >= 2:
            logger.info(f"[SweepStrat] BUY after BELOW_LOWS (confirms={confirms}/3)")
            return "BUY"

    BOT_STATS["gate_stats"]["sweep_confirms_failed"] = BOT_STATS["gate_stats"].get("sweep_confirms_failed", 0) + 1
    return None
#

```

[illegible]

[illegible]

[illegible]


```

sweep: Optional[str],
sl_mult: float = 1.5,          # P0: regime-adaptive (default = NEUTRAL)
tp_mult_ratio: float = 2.0,    # P0: regime-adaptive RR target
candle_history: Optional[List[Dict[str, Any]]] = None,
metrics: Optional[Dict[str, Any]] = None, # BUG-4: needed for atr_pct_rank
regime_p: Optional[Dict[str, Any]] = None, # PHASE-3.2: needed for panic_threshold
signal: Optional[Dict[str, Any]] = None,   # NEW: for sweep_wick access
) -> Tuple[float, float]:

    is_long = direction in ("BUY", "MEAN_REVERSAL_LONG")

    tp_from_wall = (sell_walls[0] * 0.9995 if sell_walls else None) if is_long \
        else (buy_walls[0] * 1.0005 if buy_walls else None)

    def _calc_atr_from_candles(candles: list, period: int = 14) -> float:
        if len(candles) < period + 1:
            return 0.0
        trs = []
        for i in range(1, len(candles)):
            h = candles[i].get("high", 0)
            l = candles[i].get("low", 0)
            pc = candles[i - 1].get("close", 0)
            tr = max(h - l, abs(h - pc), abs(l - pc))
            trs.append(tr)
        if len(trs) < period:
            return float(np.mean(trs)) if trs else 0.0
        atr = float(np.mean(trs[:period]))
        for j in range(period, len(trs)):
            atr = (atr * (period - 1) + trs[j]) / period
        return atr

    # PHASE-3.1: Dual vol scaling – candle-based vol_ratio AND atr_pct_rank
    # atr_pct_rank is the 30-day rolling rank (0.0=quietest, 1.0=loudest).
    # High rank (panic) = tighten stops; Low rank (quiet) = relax stops slightly.
    atr_pct_rank = metrics.get("atr_pct_rank", 0.5) if metrics else 0.5
    if candle_history and len(candle_history) >= 20:
        recent_candles = list(candle_history)[-5:]
        baseline_candles = list(candle_history)[-20:]
        # FIX-P8: recent_atr always returned 0.0 because length 5 is < period+1 (15).
        # We must explicitly set period=4 for the 5-candle recent slice.
        recent_atr = _calc_atr_from_candles(recent_candles, period=4)
        baseline_atr = _calc_atr_from_candles(baseline_candles, period=14)
        vol_ratio = recent_atr / max(baseline_atr, 1e-8)
    else:
        vol_ratio = 1.0

    vol_scale = float(np.clip(vol_ratio, 0.85, 1.50))
    rank_scale = 1.0 + (atr_pct_rank - 0.5) * 0.4
    rank_scale = float(np.clip(rank_scale, 0.75, 1.25))
    adaptive_mult = sl_mult * vol_scale * rank_scale

    if strategy_type == "TREND":
        adaptive_mult = min(adaptive_mult, 2.50)

    panic_threshold = regime_p.get("panic_threshold", 5.0) if regime_p else 5.0
    if atr_pct_rank >= (1.0 - panic_threshold / 100.0):
        emergency_mult = 1.0
        logger.warning(
            f"[RiskEngine] PANIC – ATR rank={atr_pct_rank:.0%} in top {panic_threshold:.0f}% of range. "
            f"Emergency SL: {emergency_mult:.2f}× ATR (normal adaptive={adaptive_mult:.2f})"
        )
        adaptive_mult = emergency_mult

    SL_MULT = adaptive_mult

    logger.info(
        f"[RiskEngine] sl_mult={sl_mult:.2f} vol_ratio={vol_ratio:.3f} "
        f"atr_rank={atr_pct_rank:.0%} rank_scale={rank_scale:.3f} "
        f"adaptive_mult={SL_MULT:.3f} strategy={strategy_type}"
    )

    TP_MULT = tp_mult_ratio # P0: regime-adaptive (RR target from REGIME_PARAMS)

    def sl_tp(sl_dist: float) -> Tuple[float, float]:
        if is_long:
            sl = price - sl_dist
            tp_target = price + (sl_dist * TP_MULT)
            tp = tp_from_wall if (tp_from_wall and tp_from_wall > tp_target) else tp_target
        else:
            sl = price + sl_dist
            tp_target = price - (sl_dist * TP_MULT)
            tp = tp_from_wall if (tp_from_wall and tp_from_wall < tp_target) else tp_target
        return round(sl, 2), round(tp, 2)

    if strategy_type == "LIQUIDITY_SWEEP" and sweep:
        if sweep == "ABOVE_HIGHS" and sell_walls:
            sl_dist = abs(price - sell_walls[0]) + (atr * 0.5)
        elif sweep == "BELOW_LOWS" and buy_walls:
            sl_dist = abs(price - buy_walls[0]) + (atr * 0.5)
        else:

```

```
sl_dist = atr * SL_MULT
_sl_wick_override = None
if signal is not None:
    _wick = signal.get("sweep_wick", 0.0)
    if signal.get("strategy_type") == "LIQUIDITY_SWEEP" and _wick > 0:
        _buf = 0.20 * atr
        if is_long:
            _sl_wick_candidate = _wick - _buf
            if _sl_wick_candidate < price - sl_dist:
                _sl_wick_override = _sl_wick_candidate
        else:
            _sl_wick_candidate = _wick + _buf
            if _sl_wick_candidate > price + sl_dist:
                _sl_wick_override = _sl_wick_candidate
    if _sl_wick_override is not None:
        logger.info(f"[RiskEngine] Wick SL: {price - sl_dist:.2f}→{_sl_wick_override:.2f}")
        sl_dist = abs(price - _sl_wick_override)
return sl_tp(max(sl_dist, atr * SL_MULT))
elif strategy_type == "TREND":
    return sl_tp(atr * SL_MULT)
elif strategy_type == "MEAN_REVERSION":
    return sl_tp(atr * SL_MULT)
else:
    return sl_tp(price * 0.008)
```

■■■ FULL 7-STAGE SIGNAL ENGINE ■■■■

```
async def _compute_signal(
    metrics: Dict[str, Any],
    candle_history: list,
    feed_state,
    daily_loss_halt: bool,
    drawdown_halt: bool = False,
    quant=None, # PHASE-5.1: QuantEngine ref for cold-start trade count
) -> Dict[str, Any]:
    WAIT = {
        "verdict": "WAIT", "confidence": 0.0,
        "stop_loss": 0.0, "take_profit": 0.0,
        "analysis": "", "ulis_verdict": "-"
    }

    if daily_loss_halt:
        logger.warning("[RiskEngine] Daily loss limit hit – all trading halted today.")
        _gate_stats_summary("daily_loss_halt")
        return {"**WAIT, "analysis": "Daily loss limit reached. Halted."}

    if drawdown_halt:
        logger.warning("[RiskEngine] Max drawdown breached – all trading halted. Restart bot to resume.")
        _gate_stats_summary("drawdown_halt")
        return {"**WAIT, "analysis": "Max drawdown breached. Restart bot to resume."}

# ■ Capital Hard Stop ■■■■
# PATCH (2026-05-13): Lowered from $150 to $50.
# $150 was a conservative guideline. Fee math still works at $77 on 15x
# leverage – round-trip fee = 0.04% × 15 × 2 = 1.2% of equity, which is
# covered by any TP hit on a 1.8:1+ RR trade. Real risk at low equity is
# position sizing – handled by the small-account risk cap below.
_MIN_VIABLE_EQUITY = 50.0
if ACCOUNT_SIZE < _MIN_VIABLE_EQUITY:
    logger.warning(
        f"[RiskEngine] ■ Account ${ACCOUNT_SIZE:.2f} below absolute minimum "
        f"${_MIN_VIABLE_EQUITY:.0f}. All entries blocked. Deposit to resume."
    )
    _gate_stats_summary("signal_none")
    return {"**WAIT, "analysis": f"Account ${ACCOUNT_SIZE:.2f} < ${_MIN_VIABLE_EQUITY:.0f} minimum. Deposit to resume."}

# ■ Small-Account Risk Cap ■■■■
# When equity is below the recommended $150, cap risk per trade at 0.75%
# to prevent fee erosion from consecutive losses destroying the account.
# Win rate is the priority – protecting capital between signals is essential.
_RECOMMENDED_EQUITY = 150.0
_SMALL_ACCOUNT_RISK_CAP = 0.75 # % of equity – max loss ~$0.58 at $77
_small_account_mode = ACCOUNT_SIZE < _RECOMMENDED_EQUITY
if _small_account_mode:
    logger.info(
        f"[RiskEngine] Small-account mode active – risk capped at "
        f"{_SMALL_ACCOUNT_RISK_CAP}% (equity=${ACCOUNT_SIZE:.2f} < "
        f"${_RECOMMENDED_EQUITY:.0f} recommended). Win-rate focus."
    )

# PHASE-0.4: Z-Score session guard
# Z-score is statistically meaningless with fewer than 10 bars – it fits noise.
if not metrics.get("z_score_valid", True):
    _gate_stats_summary("zscores-warmup")
    return {"**WAIT, "analysis": "Session warmup: Z-Score not yet valid (<10 bars)."}

# --- AUDIT FIX 1: ATR PANIC GATE ---
atr_pct_rank = metrics.get("atr_pct_rank", 0.5)
```

[illegible]

```

        logger.info(
            f"[CandleGate] Sweep signal stale – sweep candle closed "
            f"{sweep_candle_age:s}.0f}s ago (>{max_sweep_age}s). Blocking."
        )
        BOT_STATS["gate_stats"]["candle_gate_expired"] = BOT_STATS["gate_stats"].get("candle_gate_expired", 0) + 1
        sweep = None

# Stage 4: Strategy
raw_direction: Optional[str] = None
strategy_type: str

if sweep:
    strategy_type = "LIQUIDITY_SWEEP"
    raw_direction = _strategy_liquidity_sweep(sweep, metrics)
    logger.info(f"[MetaModel] → LIQUIDITY_SWEEP (sweep={sweep})")
elif regime == "TREND":
    strategy_type = "TREND"
    raw_direction = _strategy_trend(metrics)
    logger.info(f"[MetaModel] → TREND strategy")
elif regime == "RANGE":
    strategy_type = "MEAN_REVERSION"
    raw_direction = _strategy_mean_reversion(
        metrics,
        z_threshold=regime_p["z_threshold"],
    )
    logger.info(f"[MetaModel] → MEAN_REVERSION strategy (z_thr={regime_p['z_threshold']})")
elif regime == "NEUTRAL":
    strategy_type = "MEAN_REVERSION"
    z_current = metrics.get("zScore", 0.0)
    z_prev = metrics.get("zScore_prev", z_current)
    z_slope = z_current - z_prev
    rsi = metrics.get("rsi", 50.0)
    rsi_prev = metrics.get("rsi_prev", rsi)
    rsi_prev2 = metrics.get("rsi_prev2", rsi_prev)
    rsi_trough = (rsi > rsi_prev) and (rsi_prev < rsi_prev2)
    rsi_peak = (rsi < rsi_prev) and (rsi_prev > rsi_prev2)
    z_thr = regime_p["z_threshold"]
    candidate = _strategy_mean_reversion(metrics, z_threshold=z_thr)
    if candidate == "MEAN_REVERSAL_LONG" and (z_slope <= 0 or not rsi_trough):
        candidate = None
    if candidate == "MEAN_REVERSAL_SHORT" and (z_slope >= 0 or not rsi_peak):
        candidate = None
    raw_direction = candidate
    if raw_direction:
        logger.info(f"[MetaModel] NEUTRAL -> {strategy_type} ({raw_direction}) "
            f"z={z_current:.2f} slope={z_slope:+.3f}")
    else:
        logger.debug(f"[MetaModel] NEUTRAL: no setup (z={z_current:.2f}, "
            f"slope={z_slope:+.3f}")
else:
    strategy_type = "NEUTRAL"
    raw_direction = None
    logger.info(f"[MetaModel] → WAIT (regime={regime}, no sweep)")

if raw_direction is None:
    _gate_stats_summary("signal_none")
    return {"**WAIT", "analysis": f"Regime={regime} strategy={strategy_type} – no edge."}

# --- AUDIT FIX 3: RSI EXTREMES GATE ---
rsi = metrics.get("rsi", 50.0)
is_long_dir = raw_direction in ("BUY", "MEAN_REVERSAL_LONG")
if is_long_dir and rsi > 70.0:
    logger.warning(f"[RiskEngine] ■ RSI OVERBOUGHT HALT – Blocking {raw_direction} at RSI={rsi:.1f} > 70.0")
    _gate_stats_summary("rsi_overbought")
    return {"**WAIT", "analysis": f"RSI={rsi:.1f} > 70.0 blocks {raw_direction} entry."}
if not is_long_dir and rsi < 30.0:
    logger.warning(f"[RiskEngine] ■ RSI OVERSOLD HALT – Blocking {raw_direction} at RSI={rsi:.1f} < 30.0")
    _gate_stats_summary("rsi_oversold")
    return {"**WAIT", "analysis": f"RSI={rsi:.1f} < 30.0 blocks {raw_direction} entry."}

# Stage 4b: HTF Counter-Trend Block
is_long_dir = raw_direction in ("BUY", "MEAN_REVERSAL_LONG")
is_trend_strat = strategy_type == "TREND"

if regime_p["htf_block"]: # P0: Only apply HTF block when regime says to
    if htf == "BEAR" and is_long_dir and is_trend_strat:
        logger.warning(f"[HTF] Blocking LONG trend trade – 4H trend is BEAR.")
        _gate_stats_summary("htf_counter_trend")
        return {"**WAIT", "analysis": f"HTF=BEAR blocks {raw_direction} trend entry."}
    if htf == "BULL" and not is_long_dir and is_trend_strat:
        logger.warning(f"[HTF] Blocking SHORT trend trade – 4H trend is BULL.")
        _gate_stats_summary("htf_counter_trend")
        return {"**WAIT", "analysis": f"HTF=BULL blocks {raw_direction} trend entry."}

if is_trend_strat:
    try:
        fr = float(metrics.get("funding_rate", 0.0))
        # P1-6 FIX: Old threshold (0.015%) was 5-10x below normal BTC contango
        # (typically 0.03-0.10% per 8h). It silently blocked ~70% of long entries
        # in any bullish session. New thresholds reflect genuinely crowded positions.

```

```

FUNDING_LONG_BLOCK = 0.0008 # 0.08% per 8h – clearly crowded longs
FUNDING_SHORT_BLOCK = -0.0005 # -0.05% per 8h – clearly crowded shorts
if is_long_dir and fr > FUNDING_LONG_BLOCK:
    logger.warning(f"[Anti-Squeeze] Blocking LONG: extreme funding rate {fr:.4%} > {FUNDING_LONG_BLOCK:.4%}")
    _gate_stats_summary("funding_blocks_long")
    return {"**WAIT", "analysis": f"Funding Rate {fr:.4%} > {FUNDING_LONG_BLOCK:.4%}. Blocked long."}
if not is_long_dir and fr < FUNDING_SHORT_BLOCK:
    logger.warning(f"[Anti-Squeeze] Blocking SHORT: extreme funding rate {fr:.4%} < {FUNDING_SHORT_BLOCK:.4%}")
    _gate_stats_summary("funding_blocks_short")
    return {"**WAIT", "analysis": f"Funding Rate {fr:.4%} < {FUNDING_SHORT_BLOCK:.4%}. Blocked short."}
except (ValueError, TypeError):
    pass

# FIX-P4: HTF Directional Filter for LIQUIDITY Sweep entries.
# Fix: sweep entries must align with the reversal implied by the sweep direction vs HTF.
# HTF=NEUTRAL → no bias, allow both directions.
is_sweep_strat = strategy_type == "LIQUIDITY_SWEEP" and sweep is not None
if is_sweep_strat and htf != "NEUTRAL":
    if htf == "BEAR" and is_long_dir:
        logger.warning(
            "[HTF-Sweep] BUY sweep BLOCKED – 4H BEAR trend. "
            "BELOW_LOWS into a bear trend is a continuation, not a reversal."
        )
        _gate_stats_summary("htf_counter_trend")
        return {"**WAIT", "analysis": "HTF=BEAR blocks BUY sweep (not a genuine reversal setup)."}
    if htf == "BULL" and not is_long_dir:
        logger.warning(
            "[HTF-Sweep] SELL sweep BLOCKED – 4H BULL trend. "
            "ABOVE_HIGHS into a bull trend is a continuation, not a reversal."
        )
        _gate_stats_summary("htf_counter_trend")
        return {"**WAIT", "analysis": "HTF=BULL blocks SELL sweep (not a genuine reversal setup)."}

# === DERIVATIVES GATE: Institutional signal check ===
# Fetch context (cached, so this is near-instant on most calls)
_deriv_ctx = await _derivatives_ctx.get_full_context()
_deriv_ok, _deriv_reason = await _derivatives_gate(raw_direction, _deriv_ctx)
if not _deriv_ok:
    logger.warning(f"[DerivGate] VETO: {_deriv_reason}")
    _gate_stats_summary("derivatives_veto")
    return {"**WAIT", "analysis": f"Derivatives gate veto: {_deriv_reason}"}
if _deriv_reason:
    logger.info(f"[DerivGate] PASS ({_deriv_reason})")

# FIXED: CVD gate BEFORE Bayes fusion (was after)
_pre_div_conf = metrics.get("bayesianPosterior", 0.5)
_div_conf, _div_veto = _apply_cvd_divergence_gate(raw_direction, metrics, _pre_div_conf)
if _div_veto:
    _gate_stats_summary("cvd_divergence_veto")
    return {"**WAIT", "analysis": f"CVD divergence veto: {_div_veto}"}
if abs(_div_conf - _pre_div_conf) > 0.001:
    metrics = {"**metrics", "bayesianPosterior": _div_conf}

confidence = _bayesian_fusion(metrics, raw_direction, regime, is_sweep=bool(sweep))

# --- AUDIT FIX 2: BAYES FLOOR GATE ---
if confidence < 0.50:
    logger.warning(f"[RiskEngine] ■ BAYES FLOOR HALT – Bayesian posterior {confidence:.2%} < 50%. Edge is worse than a coin")
    _gate_stats_summary("bayes_floor_veto")
    return {"**WAIT", "analysis": f"Bayesian Posterior {confidence:.2%} < 50% blocks entry."}

# Stage 5b: VP0C Proximity Confidence Boost
vpoc_boost = _vpoc_confidence_boost(price, vpoc, raw_direction)
if vpoc_boost > 0.0:
    confidence = min(1.0, confidence + vpoc_boost)
    logger.info(f"[VP0C] Near VP0C ({vpoc:.0f}). Confidence boosted by +{vpoc_boost:.2%} → {confidence:.2%}")

logger.info(f"[BayesFusion] direction={raw_direction} | P={confidence:.2%}")

# Stage 6 ★ ULIS/ALDE gate FIRST (applies confidence boost)
should_trade, confidence, ulis_verdict_str = _apply_ulis_gate(
    metrics, candle_history, feed_state, raw_direction, confidence
)

if not should_trade:
    _gate_stats_summary("ulis_veto")
    return {"**WAIT", "analysis": f"ULIS veto: {ulis_verdict_str}", "ulis_verdict": ulis_verdict_str}

# FIX: Check threshold AFTER ULIS gate using boosted confidence
# P0: Use regime-adaptive min_confidence instead of global MIN_CONFIDENCE
# PHASE-5.1: Apply cold-start discount (few trades = slightly lower bar)
# PHASE-5.2: Wire MIN_CONFIDENCE as a floor under regime threshold
total_trades = sum(quant.get_regime_trade_counts().values()) if quant else 0
cold_start_discount = COLD_START_CONFIDENCE_DISCOUNT if total_trades < COLD_START_TRADE_COUNT else 0.0
# FIX: Ensure global environment MIN_CONFIDENCE does not force an impossibly high hurdle.
regime_min_conf = max(
    regime_p["min_confidence"] - cold_start_discount,
    0.50
)
if confidence < regime_min_conf:

```

[illegible]

```

stats: dict,
quant,
executor,
):

global LAST_CASCADE_TIME, LAST_ANY_TRADE_CLOSE_TIME, LAST_TRADE_WAS_SL

LAST_ANY_TRADE_CLOSE_TIME = time.time()
LAST_TRADE_WAS_SL = (pnl < 0)

stats["daily_pnl"] = stats.get("daily_pnl", 0.0) + pnl
stats["session_pnl"] = stats.get("session_pnl", 0.0) + pnl

quant.update_win_rate(won=(pnl >= 0), regime=regime)

if pnl < 0:
    LAST_CASCADE_TIME = time.time()
    stats["consecutive_losses"] = stats.get("consecutive_losses", 0) + 1
    now_t = time.time()
    loss_times = [t for t in stats.get("loss_times", []) if now_t - t < 1800]
    loss_times.append(now_t)
    stats["loss_times"] = loss_times
    if len(loss_times) >= 3:
        stats["cooldown_until"] = now_t + 1800
        stats["loss_times"] = []
        logger.error("[RiskManager] 3 SL exits within 30 min – 30-min cooldown active.")
        if executor.notifier:
            await executor.notifier.send_message(
                f"■ Quad-Desk CONSECUTIVE LOSS HALT\n"
                f"3 SL exits within 30 minutes. Paused 30 min."
            )
    else:
        stats["consecutive_losses"] = 0

# P0-2 FIX: Use live equity (not static boot ACCOUNT_SIZE) for risk calcs.
# After gains/losses, the absolute dollar limits must scale with real equity.
current_equity = ACCOUNT_SIZE + stats.get("session_pnl", 0.0)

max_loss_usd = current_equity * MAX_DAILY_LOSS_PCT / 100.0
if stats["daily_pnl"] < -max_loss_usd and not stats.get("daily_loss_halt"):
    stats["daily_loss_halt"] = True
    logger.warning(
        f"[RiskEngine] ■ Daily loss limit: ${stats['daily_pnl']:.2f} "
        f"(limit=-${max_loss_usd:.2f} on equity=${current_equity:.2f}). Halted until tomorrow."
    )

max_drawdown_usd = current_equity * MAX_DRAWDOWN_PCT / 100.0
if stats["session_pnl"] < -max_drawdown_usd and not stats.get("drawdown_halt"):
    stats["drawdown_halt"] = True
    logger.critical(
        f"[RiskEngine] ■ MAX DRAWDOWN BREACHED: "
        f"${stats['session_pnl']:.2f} (limit=-${max_drawdown_usd:.2f} on equity=${current_equity:.2f}). HALTED."
    )
    if executor.notifier:
        await executor.notifier.send_message(
            f"■ Quad-Desk MAX DRAWDOWN HIT\n"
            f"Session Pnl: ${stats['session_pnl']:.2f} / Limit: -${max_drawdown_usd:.2f}\n"
            f"Bot halted. Restart to resume."
        )

async def execution_loop(
    feed,
    quant: QuantEngine,
    executor: TradingExecutor,
    stats: Dict[str, Any],
):
    await executor.initialize()

    # CRIT-2 FIX: Set leverage AFTER markets are loaded (initialize() calls load_markets).
    # Pre-initialize leverage set could hit Binance with an unvalidated symbol and
    # silently fail, leaving the account at whatever leverage was last manually set.
    if executor.is_futures and not executor.dry_run:
        try:
            ccxt_symbol = executor._get_ccxt_symbol(FEED_SYMBOL)
            await executor.exchange.set_leverage(LEVERAGE, ccxt_symbol)
            logger.info(f"[Main] Futures leverage confirmed: {LEVERAGE}x on {ccxt_symbol} ✓")
        except Exception as e:
            logger.warning(f"[Main] Could not set leverage (will continue with account default): {e}")

    global ACCOUNT_SIZE, LAST_CVD, LAST_CASCADE_TIME, LAST_ANY_TRADE_CLOSE_TIME, LAST_CANDLE_TS, LAST_TRADE_WAS_SL
    global _CYCLE_ERROR_COUNT, _LAST_CYCLE_ERROR

    if not executor.dry_run:
        try:
            fetched_bal = await executor.get_usdt_balance(ACCOUNT_SIZE)
            if fetched_bal != ACCOUNT_SIZE and fetched_bal > 0:
                logger.info(f"[Main] Overriding BOT_ACCOUNT_SIZE with dynamically fetched live balance: ${fetched_bal:.2f}")
                ACCOUNT_SIZE = fetched_bal
        except Exception as e:
            logger.warning(f"[Main] Could not dynamically fetch account balance at startup: {e}")

```

```

stats["account_equity"] = ACCOUNT_SIZE + stats.get("session_pnl", 0.0)

EQUITY_MIN_TRADEABLE = 200.0
if ACCOUNT_SIZE < EQUITY_MIN_TRADEABLE:
    logger.warning(
        f"[RiskEngine] ■■ Account ${ACCOUNT_SIZE:.2f} below recommended minimum "
        f"${EQUITY_MIN_TRADEABLE:.0f}). Trade sizing may fail or be suboptimal."
    )
    if executor.notifier:
        await executor.notifier.send_message(
            f"■■ Low Equity: ${ACCOUNT_SIZE:.2f} (min recommended: ${EQUITY_MIN_TRADEABLE:.0f})"
        )

mode = "DRY-RUN" if executor.dry_run else "LIVE"
logger.info(
    f"■■■ Quad-Desk Bot ████████████████████████████████████\n"
    f"■ Exchange      : {EXCHANGE.upper()}\n"
    f"■ Symbol       : {SYMBOL}\n"
    f"■ Feed Symbol  : {FEED_SYMBOL} (Binance WS)\n"
    f"■ Mode        : {mode}\n"
    f"■ Risk/trade   : {MAX_RISK_PCT}% (${ACCOUNT_SIZE * MAX_RISK_PCT / 100:.2f} max loss)\n"
    f"■ Max daily ↓  : {MAX_DAILY_LOSS_PCT}%\n"
    f"■ Min P(edge)  : {MIN_CONFIDENCE:.0%}\n"
    f"■ ULIS Gate    : {'ON ✓' if ULIS_GATE_ENABLED else 'OFF'}\n"
    f"■ Interval     : every {ANALYSIS_INTERVAL}s\n"
    f"████████████████████████████████████████████████████████████"
)

_last_recon_ts = 0.0
_orphan_flatten_attempts: int = 0          # how many times we've tried to flatten an orphan
_last_orphan_flatten_ts: float = 0.0      # timestamp of last flatten attempt

# Startup lockout: prevent any entry for the first 90s after boot.
# The REST prefetch loads 100 historical candles, but live CVD, OFI and
# order-book readings need a few cycles to stabilise before we act on them.
_STARTUP_LOCKOUT_SECS = 90
_boot_time = time.time()
logger.info(
    f"[Main] ■ Startup lockout active – no new entries for {_STARTUP_LOCKOUT_SECS}s "
    f"While live feed settles."
)

while True:
    try:
        # ■■ Event-driven execution (Phase 6) ████████████████████████████████████
        try:
            # Wait for candle close OR timeout (whichever comes first)
            await asyncio.wait_for(
                asyncio.shield(feed.state.candle_close_event.wait()),
                timeout=float(ANALYSIS_INTERVAL)
            )
            feed.state.candle_close_event.clear()
        except (asyncio.TimeoutError, TimeoutError):
            pass # Fallback to polling if websocket hasn't fired yet

        # ■■ DIAGNOSTIC: confirm loop is alive every cycle ████████████████████
        n_candles = len(feed.state.candles)
        logger.debug(f"[Main] Cycle tick | candles={n_candles} | "
                    f"trades={len(feed.state.recent_trades)} | cvd={feed.state.cvd:.0f}")

        if n_candles < QuantEngine.MIN_CANDLES:
            logger.info(f"[Main] Warming up... {n_candles}/{QuantEngine.MIN_CANDLES} candles")
            continue

        # ■■ Startup lockout guard ████████████████████████████████████████████
        # Allow exit monitoring immediately, but block new entries until
        # the live feed has had time to build real CVD/OFI/order-book data.
        _startup_elapsed = time.time() - _boot_time
        _in_startup_lockout = _startup_elapsed < _STARTUP_LOCKOUT_SECS

        # ■■ Daily loss reset at midnight ████████████████████████████████████████
        today = date.today()
        if stats.get("_last_trade_day") != today:
            stats["_last_trade_day"] = today
            stats["daily_pnl"] = 0.0
            stats["daily_loss_halt"] = False
            # PHASE-3.1: Drawdown halt must reset daily, not require manual restart
            if stats.get("drawdown_halt"):
                stats["drawdown_halt"] = False
            stats["session_pnl"] = 0.0
            logger.warning("[RiskEngine] New day – drawdown halt LIFTED, session PnL reset.")
            # 3.6 FIX: Reset CVD anchor daily to prevent multi-day drift
            feed.state.cvd = 0.0
            LAST_CVD = 0.0
            logger.info("[RiskEngine] ■ Daily counters + CVD reset for new trading session.")
            # ■■ FIX #6: CVD Divergence Snapshot Reset (Addition B) ████████████████
            # CRITICAL: Without clearing snapshots, first divergence comparison
            # of the new day measures ΔCVD = 0 - prior_session_CVD(e.g. -8000)
            # → spurious BULLISH divergence of strength ~1.0 on first signal.
            # Also reset EMA-variance state to prevent prior-session CVD

```


[illegible]

[illegible]

```

current_time = time.time()
if not executor.dry_run and (current_time - _last_recon_ts) >= 300:
    _last_recon_ts = current_time
    try:
        if _cached_positions is None:
            _cached_positions = await executor.exchange.fetch_positions()
        positions = _cached_positions
        exchange_has_pos = any(
            abs(float(p.get("contracts", 0) or p.get("positionAmt", 0))) > 0.0001
            for p in positions
        )
        bot_has_pos = executor.active_position is not None
        if exchange_has_pos and not bot_has_pos:
            # Issue #7 FIX: Auto-close orphan position.
            # Only attempt once per 30 minutes to prevent Telegram spam.
            # After the first attempt, log a single warning and stand down
            # until the user manually resolves or the position closes.
            _now_ts = time.time()
            _time_since_last_orphan = _now_ts - _last_orphan_flatten_ts
            if _orphan_flatten_attempts == 0 or _time_since_last_orphan >= 1800:
                _orphan_flatten_attempts += 1
                _last_orphan_flatten_ts = _now_ts
                logger.critical(
                    f"[Reconciliation] Orphan position detected (attempt #{_orphan_flatten_attempts}) - "
                    "exchange has open position but bot has no tracking. "
                    "Emergency flatten initiated."
                )
            )
            try:
                await executor.notifier.send_error_alert(
                    f"■■ Orphan position detected (attempt #{_orphan_flatten_attempts}).\n"
                    "Bot has no SL/TP for this position.\n"
                    "Auto-flatten initiated. If this repeats, close manually on Binance."
                )
                await executor.emergency_flatten("Orphan position detected on reconciliation")
                logger.info("[Reconciliation] Emergency flatten call completed.")
            except Exception as flatten_err:
                logger.error(
                    f"[Reconciliation] Emergency flatten FAILED: {flatten_err}. "
                    "Position may still be open on exchange - MANUAL ACTION REQUIRED."
                )
                await executor.notifier.send_error_alert(
                    f"■ AUTO-FLATTEN FAILED: {flatten_err}\n"
                    "■ Manual intervention required on Binance."
                )
            )
        else:
            remaining = int(1800 - _time_since_last_orphan)
            logger.warning(
                f"[Reconciliation] Orphan still detected - standing down for {remaining}s "
                f"(attempted {_orphan_flatten_attempts}x already). Manual action may be needed."
            )
        )
        elif not exchange_has_pos and bot_has_pos:
            logger.warning(
                "[Reconciliation] BOT thinks position open but EXCHANGE does not. "
                "Clearing stale state."
            )
            try:
                ccxt_sym = executor.active_position.get("symbol", "")
                await executor.exchange.cancel_all_orders(ccxt_sym)
                logger.info(f"[Reconciliation] Cancelled orphaned orders for {ccxt_sym}")
            except Exception as cancel_err:
                logger.warning(f"[Reconciliation] Could not cancel orphaned orders: {cancel_err}")
            executor.active_position = None
            executor.pending_order = None
        except Exception as e:
            logger.warning(f"[Reconciliation] Check failed: {e}")

# Inject CVD delta (rate-of-change) for Bayesian fusion.
if metrics is not None:
    _current_cvd = metrics.get("cvd", 0.0)
    _cvd_reset = getattr(feed.state, "_cvd_was_reset", False)
    _cvd_suppress = getattr(executor, "_cvd_reset_suppress", 0)
    if LAST_CVD == 0.0 or _cvd_reset or _cvd_suppress > 0:
        if _cvd_reset:
            feed.state._cvd_was_reset = False
            executor._cvd_reset_suppress = 2
            logger.info("[DataFeed] CVD reset detected - suppressing delta spike for 2 cycles.")
        elif _cvd_suppress > 0:
            executor._cvd_reset_suppress = _cvd_suppress - 1
        LAST_CVD = _current_cvd
        metrics["cvd_delta"] = 0.0
    else:
        metrics["cvd_delta"] = _current_cvd - LAST_CVD
        LAST_CVD = _current_cvd

# Inject per-regime win-rate priors for Bayesian fusion blending
metrics["_regime_priors"] = quant.get_all_regime_priors()
metrics["_regime_samples"] = quant.get_regime_trade_counts()
metrics["_regime_alpha"] = dict(quant._regime_alpha)
metrics["_regime_beta"] = dict(quant._regime_beta)
# Update candle-close tracker so the candle-close gate in _compute_signal

```

```

        # knows how old the current candle is.
        if feed.state.candles:
            LAST_CANDLE_TS = float(feed.state.candles[-1]["time"])

    if executor.active_position:

        pos = executor.active_position
        if pos: # Might have been exited by the monitor
            pnl_pct = (
                (current_price - pos["entry_price"]) / pos["entry_price"] * 100
                if pos["side"] == "buy" else
                (pos["entry_price"] - current_price) / pos["entry_price"] * 100
            )
            logger.info(
                f"[Main] HOLDING {pos['side'].upper()} @ {pos['entry_price']:.2f}"
                f" | now={current_price:.2f} | PnL={pnl_pct:+.2f}%"
                f" | SL={pos['stop_loss']} TP={pos['take_profit']}"
            )

        continue

    if metrics is None:
        logger.warning(
            f"[Main] compute_metrics() returned None "
            f"(candles={len(feed.state.candles)}, trades={len(feed.state.recent_trades)}). "
            "Skipping signal evaluation this cycle."
        )
        continue

    logger.info(
        f"[Metrics] P={metrics['price']:.2f} | "
        f"RSI={metrics['rsi']:.1f} | Z={metrics['zScore']:.2f} | "
        f"Bayes={metrics['bayesianPosterior']:.2%} | Skew={metrics['skewness']:.3f} | "
        f"OFI={metrics['ofi']:.1f} | CVD={metrics['cvd']:.0f} | "
        f"ATR={metrics.get('atr', 0):.2f}({metrics.get('atr_pct', 0):.2%}) | "
        f"Tape={metrics['tapeSpeed']}/{metrics['tapeDominant']}"
    )

    # PHASE-0.2: Warn if aggTrade stream may be stale – tape metrics unreliable
    if not metrics.get("trade_buffer_healthy", True):
        logger.warning(
            f"[DataFeed] aggTrade stream stale or starved – "
            f"last trade {time.time() - feed.state._last_trade_ts:.0f}s ago, "
            f"buffer={len(feed.state.recent_trades)} trades. Tape/CVD metrics unreliable."
        )

    # Stages 2-7: Full signal engine
    # HIGH-4 FIX: Reset consecutive_losses when a cooldown expires so the
    # bot gets a clean slate after its penalty period. Without this reset,
    # two losses immediately after the cooldown trigger another 2-hour pause.
    if time.time() >= stats.get("cooldown_until", 0.0) and stats.get("_was_in_cooldown", False):
        stats["consecutive_losses"] = 0
        stats["_was_in_cooldown"] = False
        logger.info("[RiskManager] Consecutive-loss cooldown expired. Counter reset.")

    if time.time() < stats.get("cooldown_until", 0.0):
        stats["_was_in_cooldown"] = True
        verdict_json = {
            "verdict": "WAIT",
            "confidence": 0.0,
            "stop_loss": 0.0,
            "take_profit": 0.0,
        }
        "analysis": f"Consecutive Loss Cooldown active. Resumes at {time.strftime('%H:%M:%S', time.localtime(stats['cooldown_until']))}"
        "ulis_verdict": "-"
    else:
        verdict_json = await _compute_signal(
            metrics,
            candle_history=feed.state.candles,
            feed_state=feed.state,
            daily_loss_halt=stats.get("daily_loss_halt", False),
            drawdown_halt=stats.get("drawdown_halt", False),
            quant=quant,
        )

    action = verdict_json.get("verdict", "WAIT")
    conf = float(verdict_json.get("confidence", 0))
    stop_loss = verdict_json.get("stop_loss", 0)
    take_profit = verdict_json.get("take_profit", 0)
    analysis = verdict_json.get("analysis", "")
    ulis_str = verdict_json.get("ulis_verdict", "-")

    stats["last_signal"] = action
    stats["last_ulis"] = ulis_str

    # ■■■ Panic trigger: ULIS AVOID/UNWIND while holding a position ■■■
    # MED-5 FIX: Use `had_position_before_exit` (cached before exit check),
    # not executor.active_position (may be None if SL fired this same cycle).
    # Without this, a simultaneous SL+ULIS-AVOID silently skips the panic lock.

```



```

testnet=TESTNET,
dry_run=DRY_RUN,
exchange_id=EXCHANGE,
tg_token=TG_BOT_TOKEN,
tg_chat_id=TG_CHAT_ID
)

# NEW: Real-time fill detection via user data stream
async def _on_ws_fill(pnl: float):

    async with executor._position_lock:
        if executor.active_position:
            pos_regime = executor.active_position.get("regime", "NEUTRAL")
            logger.info(f"[WsFill] Position closed via ORDER_TRADE_UPDATE PnL=${pnl:.2f}")
            await _process_exit(pnl, pos_regime, BOT_STATS, quant, executor)
            BOT_STATS["active_position"] = None

if not DRY_RUN and BINANCE_API_KEY:
    asyncio.create_task(
        feed.run_user_data_stream(BINANCE_API_KEY, _on_ws_fill)
    )

# PHASE-4.2: Seed HMM observations from REST historical candles on startup
# This gives the HMM meaningful priors from the first cycle instead of
# requiring ~7 minutes of uniform posterior before it produces useful regimes.
_seeded = False
try:
    import numpy as np
    rest_candles = await feed._fetch_historical_candles_rest()
    if rest_candles and len(rest_candles) >= 20:
        closes = np.array([c["close"] for c in rest_candles], dtype=float)
        highs = np.array([c["high"] for c in rest_candles], dtype=float)
        lows = np.array([c["low"] for c in rest_candles], dtype=float)
        vols = np.array([c["volume"] for c in rest_candles], dtype=float)
        for i in range(20, len(rest_candles)):
            window = closes[max(0, i-50):i+1]
            atr_pct = (highs[i] - lows[i]) / closes[i] if closes[i] > 0 else 0.005
            z_approx = abs((closes[i] - window.mean()) / (window.std() + 1e-9))
            tape_proxy = "NORMAL"
            _hmm_classifier.classify(atr_pct, z_approx, tape_proxy, atr_pct_rank=0.5)
        logger.info(f"[HMM] Seeded with {len(rest_candles)-20} historical observations from REST candles.")
        _seeded = True

    # FIX-P1: Pre-seed quant._atr_history from REST candles (audit Problem 1).
    # Without this, _atr_history starts EMPTY. The first live ATR appended is
    # simultaneously min AND max → np.mean(arr <= atr) = 1.0 = 100% rank always.
    # This triggers PANIC SL (1.0×ATR not 1.43×ATR) on EVERY single entry.
    # Fix: compute rolling Wilder ATR at each historical step and seed the deque.
    _atr_seeded = 0
    for i in range(15, len(rest_candles)):
        _h = highs[max(0, i-43):i+1]
        _l = lows[max(0, i-43):i+1]
        _c = closes[max(0, i-43):i+1]
        if len(_c) >= 15:
            _pc = _c[-1]
            _tr = np.maximum(_h[1:]-_l[1:], np.maximum(np.abs(_h[1:]-_pc), np.abs(_l[1:]-_pc)))
            if len(_tr) >= 14:
                _av = float(np.mean(_tr[:14]))
                for _j in range(14, len(_tr)):
                    _av = (_av * 13 + float(_tr[_j])) / 14
                quant._atr_history.append(_av)
            _atr_seeded += 1
    logger.info(
        f"[ATR] Pre-seeded _atr_history: {_atr_seeded} values "
        f"(deque={len(quant._atr_history)}). "
        "atr_pct_rank now meaningful from first live cycle."
    )
except Exception as e:
    logger.warning(f"[HMM] HMM seeding from history failed (will use cold-start): {e}")

# NOTE (CRIT-2 FIX): set_leverage was moved to execution_loop() so it runs
# AFTER executor.initialize() has loaded markets. Setting leverage before

loop = asyncio.get_running_loop()
shutdown_event = asyncio.Event()

def _handle_signal():
    logger.info("[Main] Shutdown signal received.")
    shutdown_event.set()

for sig in (signal.SIGINT, signal.SIGTERM):
    try:
        loop.add_signal_handler(sig, _handle_signal)
    except NotImplementedError:
        pass

def _task_death_callback(task: asyncio.Task):
    try:
        exc = task.exception()

```

[illegible]


```
# █ CVD Post-Reset Countdown (Ghost Signal Fix 2026-05-13) ██████████
# Suppresses divergence detection for N cycles after the daily CVD reset.
# Without this the reset artifact triggers a spurious BEARISH divergence
# that persists for 100+ candles before expiring naturally (~22 min of
# false bearish bias). Set to 0 at boot; reset_cvd_divergence_state()
# sets it to 5 whenever the daily hook fires.
self._cvd_post_reset_cooldown: int = 0

# █ Session VWAP Anchor (Phase 2) ████████████████████████████████████
self._session_vwap_num: float = 0.0
self._session_vwap_den: float = 0.0
self._session_date = None

# Persistence path – survives Railway restarts if /tmp is mounted
self._persist_path = os.environ.get("BOT_STATE_PATH", "/tmp/quad_bot_state.json")
self._load_state()

# -----
# Public: update win-rate tracker after each trade
# -----
def update_win_rate(self, won: bool, regime: str = "NEUTRAL"):
    if won:
        self._alpha += 1.0
        self._regime_alpha[regime] = self._regime_alpha.get(regime, 1.0) + 1.0
        self._consecutive_losses = 0 # reset streak on any win
    else:
        self._beta += 1.0
        self._regime_beta[regime] = self._regime_beta.get(regime, 1.0) + 1.0
        self._consecutive_losses = getattr(self, "_consecutive_losses", 0) + 1
        # Prior-softening: after 3+ consecutive losses the Beta prior has drifted
        # too bearish to recover naturally. Pull it 15% toward Beta(5,5) each loss
        # so the bot can re-engage after a bad streak without a full cold-start reset.
        if self._consecutive_losses >= 3:
            _target = 5.0
            self._alpha = self._alpha * 0.85 + _target * 0.15
            self._beta = self._beta * 0.85 + _target * 0.15
            logger.warning(
                f"[QuantEngine] Consecutive loss #{self._consecutive_losses} - "
                f"softening prior toward Beta(5,5): "
                f"α={self._alpha:.1f} β={self._beta:.1f} "
                f"P(bull)={self._alpha/(self._alpha+self._beta):.1%}"
            )
        self._regime_trade_count[regime] = self._regime_trade_count.get(regime, 0) + 1
        p_bull = self._alpha / (self._alpha + self._beta)
        n = self._alpha + self._beta - 2
        logger.info(
            f"[QuantEngine] Win-rate updated: α={self._alpha:.0f} β={self._beta:.0f} "
            f"→ P(bull)={p_bull:.2%} | regime={regime} (n={n:.0f} trades)"
        )
    self._save_state()

def get_regime_trade_counts(self) -> dict:
    return dict(self._regime_trade_count)

def reset_cvd_divergence_state(self) -> None:

    self._cvd_candle_snapshots.clear()
    self._last_snapped_candle_ts = 0.0
    self._cvd_delta_ewma_mu = 0.0
    self._cvd_delta_ewma_var = 1.0
    self._cvd_delta_ewma_n = 0
    self._cvd_div_streak_dir = "NONE"
    self._cvd_div_streak_count = 0
    # Suppress divergence for 5 candles post-reset to prevent ghost bearish signal
    self._cvd_post_reset_cooldown = 5
    logger.info("[QuantEngine] CVD divergence state cleared (daily reset hook). "
               "Divergence suppressed for 5 candles to prevent ghost signal.")

# -----
# P1: Per-regime win rate accessor
# -----
def get_all_regime_priors(self) -> dict:

    result = {}
    for regime in ("RANGE", "NEUTRAL", "TREND", "LIQUIDITY"):
        a = self._regime_alpha.get(regime, 1.0)
        b = self._regime_beta.get(regime, 1.0)
        result[regime] = a / (a + b) # Beta distribution mean
    return result

# -----
# State Persistence: Firestore primary, /tmp/ file fallback
# Issue #3 FIX: /tmp/ is wiped on every Railway container restart.
# Firestore survives indefinitely. Load order on boot:
#   1. Firestore (botState/quantEngine document)
#   2. /tmp/ local file (fast path for same-container warm restarts)
#   3. Fresh Beta(5,5) prior (true cold start)
# -----
def _save_state(self):
    import threading
```

[illegible]

```
# ■■ 2. Try /tmp/ local file ████████████████████████████████████████████████████
try:
    if os.path.exists(self._persist_path):
        with open(self._persist_path) as f:
            data = json.load(f)
            self._alpha = float(data.get("alpha", 5.0))
            self._beta = float(data.get("beta", 5.0))
            saved_ra = data.get("regime_alpha", {})
            saved_rb = data.get("regime_beta", {})
            saved_rc = data.get("regime_count", {})
            for r in ("RANGE", "NEUTRAL", "TREND", "LIQUIDITY"):
                self._regime_alpha[r] = float(saved_ra.get(r, 5.0))
                self._regime_beta[r] = float(saved_rb.get(r, 5.0))
                self._regime_trade_count[r] = int(saved_rc.get(r, 0))

            self._session_vwap_num = float(data.get("session_vwap_num", 0.0))
            self._session_vwap_den = float(data.get("session_vwap_den", 0.0))
            s_date = data.get("session_date")
            from datetime import datetime
            if s_date and s_date != "None":
                try:
                    self._session_date = datetime.strptime(s_date, "%Y-%m-%d").date()
                except:
                    self._session_date = None
            logger.info(
                "[QuantEngine] ■■ State loaded from /tmp/ (Firestore unavailable). "
                "Prior will be lost on next Railway deploy."
            )
            return # ← success
except Exception as e:
    logger.warning(f"[QuantEngine] /tmp/ state load error: {e}")

# ■■ 3. Cold start – fresh Beta(5,5) ████████████████████████████████████████████████████
for r in ("RANGE", "NEUTRAL", "TREND", "LIQUIDITY"):
    self._regime_alpha[r] = 5.0
    self._regime_beta[r] = 5.0
    self._regime_trade_count[r] = 0
self._alpha = 5.0
self._beta = 5.0
logger.info("[QuantEngine] ■ Cold start: Beta(5,5) uninformed prior.")

# -----
# Public API
# -----
def compute_metrics(self) -> Optional[Dict[str, Any]]:
    if len(self.state.candles) < self.MIN_CANDLES:
        return None

    c_list = self.state.candles
    closes = np.array([c['close'] for c in c_list], dtype=float)
    highs = np.array([c['high'] for c in c_list], dtype=float)
    lows = np.array([c['low'] for c in c_list], dtype=float)
    vols = np.array([c['volume'] for c in c_list], dtype=float)

    current_price = closes[-1]

    # Calculate ATR and rank early for session VWAP and fix BUG-V3-03
    atr = self._atr(highs, lows, closes)
    self._atr_history.append(atr)
    arr = np.array(list(self._atr_history))
    atr_pct_rank = float(np.mean(arr <= atr))

    skewness = self._skewness(closes)
    z_score = self._session_vwap_z(highs, lows, closes, vols, current_price, atr_pct_rank)
    zScore_prev = getattr(self, "_last_z_score", z_score)
    self._last_z_score = z_score

    # MED-1 FIX: Use cached RSI history for rsi_prev/rsi_prev2.
    # Re-slicing closes[] and recomputing _rsi() produces a different Wilder EMA
    # seed each time – not the value that was observed last cycle. Cache instead.
    rsi = self._rsi(closes)
    self._rsi_history.append(rsi)
    rsi_prev = self._rsi_history[-2] if len(self._rsi_history) >= 2 else rsi
    rsi_prev2 = self._rsi_history[-3] if len(self._rsi_history) >= 3 else rsi_prev

    tape_speed, dominant_side = self._tape_metrics()
    ofi, wall_context, all_walls_str, execution_price, valid_bids, valid_asks, nearest_bid, nearest_ask, top_bids, top_asks :
    z_ret = self._log_return_z_score(closes)
    # Lightweight regime proxy for z_vel gating: |Z_t| > 1.5 ≈ TREND-like.
    # Full HMM regime is determined later in main.py; this gives quant engine
    # a useful signal without a circular dependency on the regime classifier.
    _regime_proxy = "TREND" if abs(z_score) > 1.5 else "NEUTRAL"
    bayesian_posterior = self._bayesian(rsi, z_score, skewness, ofi,
                                         z_ret=z_ret, regime=_regime_proxy)

    cvd = self.state.cvd

    vpcv = self._volume_poc(c_list)
    funding_rate = getattr(self.state, 'funding_rate', 0.0)
    last_trade_ts = getattr(self.state, 'last_trade ts', 0.0)
```

[illegible]

```

# -----
# 1b. Log-Return Z-Score (Z-06) – momentum / breakout detection
# -----
def _log_return_z_score(self, closes: np.ndarray, window: int = 20) -> float:
    if len(closes) < window + 2:
        return 0.0
    log_rets = np.log(closes[-(window + 1):][1:] / closes[-(window + 1):][: -1])
    valid = log_rets[np.isfinite(log_rets)]
    if len(valid) < 5:
        return 0.0
    mu = float(np.mean(valid))
    sigma = float(np.std(valid))
    if sigma < 1e-10:
        return 0.0
    z_ret = (float(valid[-1]) - mu) / sigma
    return float(np.clip(z_ret, -4.0, 4.0))

# -----
# 2. VWAP-Anchored Z-Score – True Session VWAP (Phase 2)
# -----
def _session_vwap_z(
    self,
    highs: np.ndarray,
    lows: np.ndarray,
    closes: np.ndarray,
    vols: np.ndarray,
    current_price: float,
    atr_pct_rank: float = 0.5,
) -> float:
    from datetime import datetime, timezone
    today = datetime.now(timezone.utc).date()
    if self._session_date != today:
        self._session_vwap_num = 0.0
        self._session_vwap_den = 0.0
        self._session_date = today

    if len(closes) == 0:
        return 0.0

    # Update running VWAP with latest candle
    tp = float((highs[-1] + lows[-1] + closes[-1]) / 3.0)
    v = float(vols[-1])
    self._session_vwap_num += tp * v
    self._session_vwap_den += v

    if self._session_vwap_den <= 0:
        return 0.0
    vwap_session = self._session_vwap_num / self._session_vwap_den

    # Adaptive sigma window
    n_sig = max(10, int(atr_pct_rank * 40))
    if len(closes) < n_sig:
        n_sig = len(closes)

    if n_sig <= 0:
        return 0.0

    h = highs[-n_sig:]
    l = lows[-n_sig:]
    c = closes[-n_sig:]
    vv = vols[-n_sig:]

    tp_w = (h + l + c) / 3.0
    vol_sum = np.sum(vv)
    if vol_sum <= 0:
        return 0.0

    vw_var = np.sum(vv * (tp_w - vwap_session)**2) / vol_sum
    std = np.sqrt(vw_var)

    # P1-7 FIX: Guard against near-zero std during zero-volatility periods.
    MIN_STD = current_price * 0.00005 # 0.005% of current price
    if std < MIN_STD:
        return 0.0

    # t-scale correction factor
    NU_FIXED = 6.0
    t_scale = math.sqrt((NU_FIXED - 2.0) / NU_FIXED) # = sqrt(4/6) = 0.8165
    z = (current_price - vwap_session) / std
    return float(np.clip(z * t_scale, -4.0, 4.0))

# -----
# 3. RSI (14 period) – Wilder EMA smoothing (Fix #7 from audit)
# -----
def _rsi(self, closes: np.ndarray) -> float:
    period = 14
    if len(closes) < period + 1:
        return 50.0

```

```

delta = np.diff(closes)
gains = np.where(delta > 0, delta, 0.0)
losses = np.where(delta < 0, -delta, 0.0)

# Seed with simple average over first `period` bars
avg_gain = float(np.mean(gains[:period]))
avg_loss = float(np.mean(losses[:period]))

# Wilder EMA over remaining bars
for i in range(period, len(gains)):
    avg_gain = (avg_gain * (period - 1) + gains[i]) / period
    avg_loss = (avg_loss * (period - 1) + losses[i]) / period

if avg_loss < 1e-10:
    return 100.0 if avg_gain > 0 else 50.0

rs = avg_gain / avg_loss
return float(100.0 - (100.0 / (1.0 + rs)))

# -----
# 4. Tape Speed & Dominant Side (USD-normalised)
# -----
def _tape_metrics(self):
    trades = list(self.state.recent_trades)
    now_ms = time.time() * 1000

    buy_10s = sell_10s = total_60s = 0.0
    for t in trades:
        total_60s += t['usd_volume']
        age_ms = now_ms - t['time']
        if age_ms < 10_000:
            if t['side'] == 'BUY':
                buy_10s += t['usd_volume']
            else:
                sell_10s += t['usd_volume']

    total_10s = buy_10s + sell_10s
    baseline_10s = (total_60s / 60.0) * 10 if total_60s > 0 else 0.0

    is_screaming = (
        (baseline_10s > 0 and total_10s > baseline_10s * 3)
        or total_10s > 2_000_000
    )

    if len(trades) < 5:
        return "NORMAL", "NO DATA"

    tape_speed = "SCREAMING" if is_screaming else "NORMAL"
    if buy_10s > sell_10s * 1.5:
        dominant = "BUY (ASK HIT)"
    elif sell_10s > buy_10s * 1.5:
        dominant = "SELL (BID HIT)"
    else:
        dominant = "BALANCED"

    return tape_speed, dominant

# -----
# 5. LOB Walls & OFI
# -----
def _lob_metrics(self, current_price: float):
    bids = self.state.bids
    asks = self.state.asks

    MAX_DIST = 0.001
    valid_bids = {p: s for p, s in bids.items()
                  if (current_price - p) / current_price <= MAX_DIST}
    valid_asks = {p: s for p, s in asks.items()
                  if (p - current_price) / current_price <= MAX_DIST}

    ofi = sum(valid_bids.values()) - sum(valid_asks.values())

    all_sizes = list(valid_bids.values()) + list(valid_asks.values())
    median_size = float(np.median(all_sizes)) if all_sizes else 1.0
    wall_threshold = median_size * 5.0

    # PHASE-1.1: Sort by DISTANCE to price (ascending = nearest first).
    # Previously sorted by SIZE (largest first) – wrong for sweep detection.
    # Sweep = price crossing a nearby wall, not a large distant wall.
    def bid_distance(p, s):
        return (current_price - p) / current_price # fraction of price
    def ask_distance(p, s):
        return (p - current_price) / current_price

    top_bids = sorted(
        [(p, s) for p, s in valid_bids.items() if s > wall_threshold],
        key=lambda x: bid_distance(*x),
    )[:5]
    top_asks = sorted(
        [(p, s) for p, s in valid_asks.items() if s > wall_threshold],

```

[illegible]

[illegible]


```

else:
    h = highs; l = lows; c = closes
    prev_c = c[:-1]
    tr = np.maximum(h[1:] - l[1:], np.maximum(np.abs(h[1:] - prev_c), np.abs(l[1:] - prev_c)))
    if len(tr) < period:
        return float(np.mean(tr)) if len(tr) > 0 else 0.0
    atr_val = float(np.mean(tr[:period]))
    for i in range(period, len(tr)):
        atr_val = (atr_val * (period - 1) + float(tr[i])) / period
    return float(atr_val)

# -----
# 8. Volume Point of Control (VPOC)
# -----
def _volume_poc(self, candles, n: int = 24) -> Optional[float]:
    candle_list = list(candles)[-n:]
    if len(candle_list) < 2:
        return None
    price_vol: Dict[int, float] = {}
    for c in candle_list:
        bucket_size = max(10.0, round(float(c["close"]) * 0.001 / 10.0) * 10)
        bucket = int(round(float(c["close"]) / bucket_size)) * bucket_size
        price_vol[bucket] = price_vol.get(bucket, 0.0) + float(c.get("volume", 0.0))
    if not price_vol:
        return None
    return float(max(price_vol, key=price_vol.get))

# -----
# CVD Divergence Detector – Institutional Alpha v2.0
# Fixes: #1 EMA-Z normalization | #2 True ATR | #3 Swing extrema
#         #4 Volume confirmation | #5 Uncertainty penalty data
# -----
def _cvd_divergence(self, current_price: float, true_atr: float = 0.0) -> Dict[str, Any]:
    λ = 0.92

    NULL_RESULT = {
        "type": "NONE", "strength": 0.0,
        "candles_confirmed": 0, "price_delta": 0.0,
        "cvd_delta": 0.0, "lookback_k": 0,
        "vol_spike": 1.0, "detection_method": "none",
        "n_snaps": len(self._cvd_candle_snapshots),
    }

    snaps = list(self._cvd_candle_snapshots)
    n_snaps = len(snaps)

    # ■■ Post-reset cooldown ████████████████████████████████████████████████████████████
    # Suppress divergence for N candles after daily CVD reset.
    # Each call to _cvd_divergence() corresponds to one compute_metrics()
    # cycle (~15 s). At 5 candles = ~75 s of suppression after reset.
    if self._cvd_post_reset_cooldown > 0:
        self._cvd_post_reset_cooldown -= 1
        logger.debug(
            f"[CVDDiv] Post-reset cooldown: {self._cvd_post_reset_cooldown} cycles remain."
            "Returning NONE to prevent ghost signal."
        )
        NULL_RESULT["n_snaps"] = n_snaps
        return NULL_RESULT

    if n_snaps < 2:
        return NULL_RESULT

    latest = snaps[-1]

    atr_ref = max(true_atr * 1.5, current_price * 0.003, 1.0)

    all_vols = [s.get("volume", 0.0) for s in snaps]
    mean_vol = float(np.mean(all_vols)) if all_vols else 1.0
    mean_vol = max(mean_vol, 1e-8)

    if n_snaps >= 2:
        latest_delta_raw = snaps[-1]["cvd"] - snaps[-2]["cvd"]
        self._cvd_delta_ewma_mu = (λ * self._cvd_delta_ewma_mu +
                                   (1 - λ) * latest_delta_raw)
        self._cvd_delta_ewma_var = (λ * self._cvd_delta_ewma_var +
                                    (1 - λ) * (latest_delta_raw - self._cvd_delta_ewma_mu) ** 2)
        self._cvd_delta_ewma_n += 1

    ewma_std = math.sqrt(max(self._cvd_delta_ewma_var, 1.0))

    def _norm_cvd(cvd_delta_raw: float) -> float:
        z = (cvd_delta_raw - self._cvd_delta_ewma_mu) / ewma_std
        return float(np.clip(abs(math.tanh(z / 2.0)), 0.0, 1.0))

    def _norm_price(price_delta: float) -> float:
        return float(min(abs(price_delta) / atr_ref, 1.0))

    def _norm_vol(snap_vol: float) -> float:

```

```

    spike = snap_vol / mean_vol
    return float(np.clip(math.tanh(spike / 2.0), 0.0, 1.0))

def _age_weight(k: int) -> float:
    if k <= 4:
        return 1.0
    return math.exp(-0.30 * (k - 4))

def _score(price_delta: float, cvd_delta_raw: float, vol_at_extreme: float, k: int) -> float:
    nP = _norm_price(price_delta)
    nC = _norm_cvd(cvd_delta_raw)
    nV = _norm_vol(vol_at_extreme)
    age = _age_weight(k)
    return age * (0.35 * nP + 0.45 * nC + 0.20 * nV)

swing_bull_result = None
swing_bear_result = None

if n_snaps >= 3:
    swing_low_idx = int(np.argmin([s["low"] for s in snaps]))
    swing_low_snap = snaps[swing_low_idx]
    k_swing_bull = n_snaps - 1 - swing_low_idx

    if (swing_low_idx < n_snaps - 1 and
        latest["low"] > swing_low_snap["low"] and
        latest["cvd"] > swing_low_snap["cvd"]):
        price_delta_mag = abs(latest["low"] - swing_low_snap["low"])
        cvd_delta_raw = latest["cvd"] - swing_low_snap["cvd"]
        vol_at_low = swing_low_snap.get("volume", mean_vol)
        s = _score(price_delta_mag, cvd_delta_raw, vol_at_low, k_swing_bull)
        swing_bull_result = (s, k_swing_bull, price_delta_mag,
                             cvd_delta_raw, vol_at_low / mean_vol)

    swing_high_idx = int(np.argmax([s["high"] for s in snaps]))
    swing_high_snap = snaps[swing_high_idx]
    k_swing_bear = n_snaps - 1 - swing_high_idx

    if (swing_high_idx < n_snaps - 1 and
        latest["high"] < swing_high_snap["high"] and
        latest["cvd"] < swing_high_snap["cvd"]):
        price_delta_mag = abs(latest["high"] - swing_high_snap["high"])
        cvd_delta_raw = swing_high_snap["cvd"] - latest["cvd"]
        vol_at_high = swing_high_snap.get("volume", mean_vol)
        s = _score(price_delta_mag, cvd_delta_raw, vol_at_high, k_swing_bear)
        swing_bear_result = (s, k_swing_bear, price_delta_mag,
                             cvd_delta_raw, vol_at_high / mean_vol)

max_k = min(8, n_snaps - 1)
seq_bull_best = (0.0, 0, 0.0, 0.0, 1.0)
seq_bear_best = (0.0, 0, 0.0, 0.0, 1.0)
bull_confirms = 0
bear_confirms = 0

for k in range(1, max_k + 1):
    prior = snaps[-(k + 1)]

    if latest["low"] < prior["low"] and latest["cvd"] > prior["cvd"]:
        bull_confirms += 1
        pd = prior["low"] - latest["low"]
        cd = latest["cvd"] - prior["cvd"]
        vl = prior.get("volume", mean_vol)
        s = _score(pd, cd, vl, k)
        if s > seq_bull_best[0]:
            seq_bull_best = (s, k, pd, cd, vl / mean_vol)

    if latest["high"] > prior["high"] and latest["cvd"] < prior["cvd"]:
        bear_confirms += 1
        pd = latest["high"] - prior["high"]
        cd = prior["cvd"] - latest["cvd"]
        vl = prior.get("volume", mean_vol)
        s = _score(pd, cd, vl, k)
        if s > seq_bear_best[0]:
            seq_bear_best = (s, k, pd, cd, vl / mean_vol)

best_bull_s, best_bull_k, best_bull_pd, best_bull_cd, best_bull_vs = 0.0, 0, 0.0, 0.0, 1.0
best_bull_meth = "none"
if swing_bull_result and swing_bull_result[0] >= seq_bull_best[0]:
    best_bull_s, best_bull_k, best_bull_pd, best_bull_cd, best_bull_vs = swing_bull_result
    best_bull_meth = "swing_extrema"
elif seq_bull_best[0] > 0.0:
    best_bull_s, best_bull_k, best_bull_pd, best_bull_cd, best_bull_vs = seq_bull_best
    best_bull_meth = "sequential"

best_bear_s, best_bear_k, best_bear_pd, best_bear_cd, best_bear_vs = 0.0, 0, 0.0, 0.0, 1.0
best_bear_meth = "none"
if swing_bear_result and swing_bear_result[0] >= seq_bear_best[0]:
    best_bear_s, best_bear_k, best_bear_pd, best_bear_cd, best_bear_vs = swing_bear_result
    best_bear_meth = "swing_extrema"
elif seq_bear_best[0] > 0.0:
    best_bear_s, best_bear_k, best_bear_pd, best_bear_cd, best_bear_vs = seq_bear_best

```

[illegible]

Trade Execution Logic

[illegible]

[illegible]

```

if self.exchange_id != "coinbase":
    return symbol

# Normalize: ensure we return ccxt unified format (BTC/USDC, BTC/USD, etc.)
symbol = symbol.strip().upper()

# If already in BTC/USDC format, return as-is
if "/" in symbol:
    return symbol

# If in BTC-USDC format, convert to BTC/USDC
if "-" in symbol:
    return symbol.replace("-", "/")

# BTCUSDT style → BTC/USDT
# Common quote currencies in order of descending length (avoid partial match)
for quote in ("USDT", "USDC", "USD", "BTC", "ETH", "BNB"):
    if symbol.endswith(quote):
        base = symbol[:-len(quote)]
        return f"{base}/{quote}"

# Fallback – return with / added if possible
return symbol

# -----
# Lifecycle
# -----
async def initialize(self):

    if self.dry_run and not self.exchange.apiKey:
        logger.info("[Executor] Skipping market load in keyless dry-run mode.")
        return

    env = "TESTNET" if self.testnet else "LIVE"
    exch = self.exchange_id.upper()

    # --- Retry loop: up to 3 attempts with exponential back-off ---
    last_exc: Optional[Exception] = None
    for attempt in range(3):
        try:
            await self.exchange.load_markets()
            logger.info(f"[Executor] Connected to {exch} {env} ✓ (attempt {attempt + 1})")

            # --- Boot Position Check (no reconciliation) ---
            # If an open position exists on the exchange, alert via Telegram
            # and continue trading normally. We do NOT reconcile because the
            # SL/TP would be guessed from an ATR proxy and could be dangerously wrong.
            # User commits to closing all trades before starting the bot.
            try:
                positions = await self.exchange.fetch_positions()
                open_found = False
                for pos in positions:
                    qty = float(pos.get("contracts", 0) or pos.get("positionAmt", 0))
                    if abs(qty) > 0.0001:
                        open_found = True
                        side = "buy" if qty > 0 else "sell"
                        entry_p = float(pos.get("entryPrice", 0))
                        warn_msg = (
                            f"■■■ Quad-Desk BOOT WARNING\n"
                            f"Open position found: {side.upper()} {abs(qty)} {pos.get('symbol')} @ {entry_p:.2f}\n"
                            f"Bot is starting normally but has NO internal SL/TP for this position.\n"
                            f"Please verify on Binance – this position is NOT tracked by the bot."
                        )
                        logger.warning(f"[Executor] {warn_msg}")
                        if self.notifier:
                            await self.notifier.send_message(warn_msg)
                if not open_found:
                    logger.info("[Executor] Boot clean – no open positions found on exchange.")
            except Exception as e:
                logger.warning(f"[Executor] Could not check positions on boot: {e}")

            # Send startup status notification
            try:
                mode = "DRY-RUN ■" if self.dry_run else "LIVE ■"
                await self.notifier.send_message(
                    f"■■■ Quad-Desk {mode} STARTED\n"
                    f"Exchange: {self.exchange_id.upper()} | "
                    f"Maker fee: {self.MAKER_FEE:.3%}"
                )
            except Exception:
                pass

            return # success – exit initialize
        except Exception as e:
            last_exc = e
            wait_s = 2 ** attempt # 1s, 2s, 4s
            logger.warning(
                f"[Executor] load_markets attempt {attempt + 1}/3 failed: {e}. "
                f"{'Retrying in ' + str(wait_s) + 's...' if attempt < 2 else 'All retries exhausted.'}"
            )

```

```

    )
    if attempt < 2:
        import asyncio as _asyncio
        await _asyncio.sleep(wait_s)

# All retries failed – inject minimal market fallback to prevent order failures
logger.warning(
    f"[Executor] Could not load {exch} markets after 3 attempts ({last_exc}). "
    "Injecting minimal market fallback – bot will attempt to continue."
)
self._inject_fallback_markets()

def _inject_fallback_markets(self):
    if not hasattr(self.exchange, 'markets') or self.exchange.markets is None:
        self.exchange.markets = {}
    if not hasattr(self.exchange, 'symbols') or self.exchange.symbols is None:
        self.exchange.symbols = []

    if self.exchange_id == "coinbase":
        self.exchange.markets['BTC/USDC'] = {
            'id': 'BTC-USDC', 'symbol': 'BTC/USDC', 'base': 'BTC', 'quote': 'USDC',
            'precision': {'amount': 0.00000001, 'price': 0.01},
            'limits': {'amount': {'min': 0.000001, 'max': 1000}, 'price': {'min': 0.01, 'max': 1000000}, 'cost': {'min': 1.0}},
            'active': True, 'type': 'spot', 'spot': True, 'margin': False, 'contract': False
        }
        self.exchange.markets['BTC/USD'] = {
            'id': 'BTC-USD', 'symbol': 'BTC/USD', 'base': 'BTC', 'quote': 'USD',
            'precision': {'amount': 0.00000001, 'price': 0.01},
            'limits': {'amount': {'min': 0.000001, 'max': 1000}, 'price': {'min': 0.01, 'max': 1000000}, 'cost': {'min': 1.0}},
            'active': True, 'type': 'spot', 'spot': True, 'margin': False, 'contract': False
        }
        if 'BTC/USDC' not in self.exchange.symbols: self.exchange.symbols.append('BTC/USDC')
        if 'BTC/USD' not in self.exchange.symbols: self.exchange.symbols.append('BTC/USD')

    elif self.is_futures:
        self.exchange.markets['BTC/USDT:USDT'] = {
            'id': 'BTCUSDT', 'symbol': 'BTC/USDT:USDT', 'base': 'BTC', 'quote': 'USDT', 'settle': 'USDT',
            'precision': {'amount': 0.001, 'price': 0.1},
            'limits': {'amount': {'min': 0.001, 'max': 1000}, 'price': {'min': 0.1, 'max': 1000000}},
            'active': True, 'type': 'future', 'spot': False, 'margin': False, 'contract': True
        }
        if 'BTC/USDT:USDT' not in self.exchange.symbols: self.exchange.symbols.append('BTC/USDT:USDT')

async def close(self):
    await self.exchange.close()

# -----
# Firestore trade logger
# -----
def _log_trade(self, symbol: str, side: str, verdict: str,
               entry: float, stop_loss: float, take_profit: float,
               ulis_verdict: str = "",
               metrics: dict = None,
               signal: dict = None):

    db = heartbeat.get_db()
    if db is None:
        return
    try:
        from firebase_admin import firestore as fs
        m = metrics or {}
        s = signal or {}
        doc = {
            "symbol": symbol,
            "side": side,
            "verdict": verdict,
            "entry_price": entry,
            "stop_loss": stop_loss,
            "take_profit": take_profit,
            "timestamp": fs.SERVER_TIMESTAMP,
            "mode": "DRY-RUN" if self.dry_run else "LIVE",
            "exchange": self.exchange_id,
            "ulis_verdict": ulis_verdict,
            "ts_ms": int(time.time() * 1000),
            # Signal attribution chain (5.6)
            "regime": m.get("regime", "UNKNOWN"),
            "strategy_type": s.get("strategy_type", "UNKNOWN"),
            "z_score": round(m.get("zScore", 0.0), 4),
            "rsi": round(m.get("rsi", 50.0), 2),
            "ofi_tanh": round(m.get("ofi", 0.0), 4), # tanh (-1,+1)
            "bayesian": round(m.get("bayesianPosterior", 0.5), 4),
            "confidence": round(float(s.get("confidence", 0.0)), 4),
            "atr_pct": round(m.get("atr_pct", 0.0), 6),
            "skewness": round(m.get("skewness", 0.0), 4),
        }
        res = db.collection("botTrades").add(doc)
        doc_id = res[1].id
        logger.info(f"[Executor] Trade logged to Firestore (ID: {doc_id}) ✓")
        return doc_id

```

```

except Exception as e:
    logger.warning(f"[Executor] Failed to log trade to Firestore: {e}")
    return None

def _update_trade_exit(self, doc_id: str, exit_price: float, pnl: float):
    if not doc_id:
        return
    db = heartbeat.get_db()
    if db is None:
        return
    try:
        from firebase_admin import firestore as fs
        result = "WIN" if pnl > 0 else "LOSS"
        db.collection("botTrades").document(doc_id).update({
            "exit_price": exit_price,
            "pnl": pnl,
            "result": result,
            "exit_ts": fs.SERVER_TIMESTAMP,
            "exit_ts_ms": int(time.time() * 1000)
        })
        logger.info(f"[Executor] Trade {doc_id} exit updated in Firestore ✓")
    except Exception as e:
        logger.warning(f"[Executor] Failed to update trade exit for {doc_id}: {e}")

# -----
# Balance
# -----
async def get_usdt_balance(self, account_size: float = 100.0) -> float:
    if self.dry_run:
        return account_size
    try:
        bal = await self.exchange.fetch_balance()
        free = bal.get("free", {})
        total = float(free.get("USDT", 0.0)) + float(free.get("USD", 0.0)) + float(free.get("USDC", 0.0))
        if total <= 0:
            return account_size
        return total
    except Exception as e:
        logger.warning(f"[Executor] get_usdt_balance fallback: {e}")
        return account_size

async def get_btc_balance(self) -> float:
    if self.dry_run: return 0.0
    try:
        bal = await self.exchange.fetch_balance()
        return float(bal.get("free", {}).get("BTC", 0.0))
    except Exception: return 0.0

async def get_total_equity(self, current_price: float, account_size: float = 100.0) -> float:
    if self.dry_run: return account_size
    try:
        bal = await self.exchange.fetch_balance()
        free = bal.get("free", {})

        cash = float(free.get("USDT", 0.0)) + float(free.get("USD", 0.0)) + float(free.get("USDC", 0.0))
        crypto_val = float(free.get("BTC", 0.0)) * current_price

        total = cash + crypto_val
        if total <= 5.0: # If effectively zero, fall back
            return account_size

        logger.info(f"[Executor] Equity: Cash=${cash:.2f} + BTC_Val=${crypto_val:.2f} → Total=${total:.2f}")
        return total
    except Exception as e:
        err_msg = f"Failed to calculate total equity: {e}"
        logger.error(f"[Executor] {err_msg}")
        await self.notifier.send_error_alert(err_msg)
        return account_size

# -----
# Position sizing
# -----
def calculate_position_size(
    self,
    current_price: float,
    stop_loss: float,
    equity: float,
    max_risk_pct: float,
) -> float:
    risk_usd = equity * (max_risk_pct / 100.0)
    distance = abs(current_price - stop_loss)
    if distance <= 0 or current_price <= 0:
        return 0.0
    return risk_usd / distance

```



```

# -----
# Signal execution
# -----
async def execute_signal(
    self,
    symbol: str,
    current_price: float,
    signal: Dict[str, Any],
    max_risk_pct: float,
    account_size: float = 100.0,
    ulis_verdict: str = "",
):
    verdict = signal.get("verdict", "WAIT")
    confidence = float(signal.get("confidence", 0))
    stop_loss = float(signal.get("stop_loss", 0))
    take_profit = float(signal.get("take_profit", 0))

    # Guard rails
    if "WAIT" in verdict:
        return

    if self.active_position is not None:
        logger.info("[Executor] Already in a position. Skipping new entry.")
        return

    # Check if previous order is still pending (unfilled)
    if self.pending_order is not None:
        # MED-5 FIX: Auto-expire stale pending_order to prevent permanent lock.
        if time.time() > self.pending_order.get("expires_at", 0):
            logger.warning(
                f"[Executor] Pending order {self.pending_order['id']} TTL expired - "
                "clearing stale state. Verify on exchange if order was filled."
            )
            self.pending_order = None
        else:
            logger.info(f"[Executor] Pending order {self.pending_order['id']} still unfilled. Rejecting new signal.")
            return

    if stop_loss <= 0 or take_profit <= 0:
        logger.warning("[Executor] Invalid SL/TP. Aborting.")
        return

    side = "buy" if ("BUY" in verdict or "LONG" in verdict) else "sell"
    # AUDIT FIX #6: Use estimated fill price (bid + spread proxy) for SL check.
    # current_price is the bid. BUY orders fill at the ask (~bid + 2bp).
    # Without this, a SL set between bid and ask passes the check but
    # gets immediately hit at fill.
    fill_price_est = current_price * 1.0002 if side == "buy" else current_price * 0.9998
    if side == "buy" and stop_loss >= fill_price_est:
        logger.warning(f"[Executor] SL {stop_loss} ≥ est. fill {fill_price_est:.2f} for BUY. Aborting.")
        return
    if side == "sell" and stop_loss <= fill_price_est:
        logger.warning(f"[Executor] SL {stop_loss} ≤ est. fill {fill_price_est:.2f} for SELL. Aborting.")
        return

    # 1. Calculate Available Equity for accurate risk sizing.
    # Spot: Use Total Equity (Cash + BTC) to size based on full portfolio.
    if self.is_futures:
        equity = await self.get_usdt_balance(account_size)
    else:
        equity = await self.get_total_equity(current_price, account_size)

    if side == "buy":
        usdc_equity = equity
        if usdc_equity < 5:
            err = f"Insufficient USDC (${usdc_equity:.2f}) to open LONG. Min $5 required."
            logger.warning(f"[Executor] {err}")
            await self.notifier.send_error_alert(err)
            return
    else:
        if self.is_futures:
            usdc_equity = equity
            if usdc_equity < 5:
                err = f"Insufficient USDC margin (${usdc_equity:.2f}) to open SHORT on Binance Futures. Min $5 required."
                logger.warning(f"[Executor] {err}")
                await self.notifier.send_error_alert(err)
                return
        else:
            # Spot: Need BTC in account to sell
            btc_bal = await self.get_btc_balance()
            if btc_bal <= 0.00001:
                err = f"Aborting SELL signal: No BTC balance available to sell on spot account."
                logger.warning(f"[Executor] {err}")
                await self.notifier.send_error_alert(err)
                return

    raw_size = self.calculate_position_size(current_price, stop_loss, equity, max_risk_pct)
    if raw_size <= 0.0:
        logger.warning("[Executor] Calculated position size is 0. Aborting.")
        return

```

```

# FIX 16: Minimum validation with operator alert
MIN_NOTIONAL_USDT = 105.0 # Binance BTCUSDT min $100 + 5% buffer
MIN_QTY_BTC = 0.001 # Binance BTCUSDT minimum lot size

if raw_size * current_price < MIN_NOTIONAL_USDT:
    needed_acct = (MIN_NOTIONAL_USDT * abs(current_price - stop_loss)) / (max_risk_pct / 100.0)
    logger.error(
        f"[Executor] Account ${equity:.0f} too small. "
        f"Risk-correct notional=${raw_size*current_price:.2f} < Binance min ${MIN_NOTIONAL_USDT}. "
        f"Needs ~${needed_acct:.0f} account. ABORTING order. "
        "Set BOT_ACCOUNT_SIZE=1500 in Railway to resolve."
    )
    if self.notifier:
        await self.notifier.send_message(
            f"Account too small for execution. "
            f"Current: ${equity:.0f}. Required: ~${needed_acct:.0f}."
        )
    return # Abort - do not scale up

# Translate symbol to exchange format (unified CCXT symbol)
if self.exchange_id == "coinbase":
    ex_symbol = self._to_exchange_symbol(symbol)
else:
    ex_symbol = self._get_ccxt_symbol(symbol)

fmt_size = float(self.exchange.amount_to_precision(ex_symbol, raw_size))
if fmt_size < MIN_QTY_BTC:
    logger.warning(f"[Executor] fmt_size {fmt_size} < min qty {MIN_QTY_BTC}. Aborting.")
    return

# For Coinbase, ensure the symbol is in the markets cache
if self.exchange_id == "coinbase":
    if ex_symbol not in self.exchange.markets or self.exchange.markets.get(ex_symbol) is None:
        # Try to add it to markets cache if missing
        logger.warning(f"[Executor] Symbol {ex_symbol} not in markets cache. Attempting to register...")
        if not hasattr(self.exchange, 'markets') or self.exchange.markets is None:
            self.exchange.markets = {}

        # Use default market spec for BTC/USDC or BTC/USD
        if "BTC" in ex_symbol.upper() and "USDC" in ex_symbol.upper():
            self.exchange.markets['BTC/USDC'] = {
                'id': 'BTC-USDC', 'symbol': 'BTC/USDC', 'base': 'BTC', 'quote': 'USDC',
                'precision': {'amount': 0.00000001, 'price': 0.01},
                'limits': {'amount': {'min': 0.00001, 'max': 1000}, 'price': {'min': 0.01, 'max': 1000000}, 'cost': {'min': 100}},
                'active': True, 'type': 'spot', 'spot': True, 'margin': False, 'contract': False
            }
            logger.info(f"[Executor] Registered BTC/USDC to markets cache.")
        elif "BTC" in ex_symbol.upper() and "USD" in ex_symbol.upper():
            self.exchange.markets['BTC/USD'] = {
                'id': 'BTC-USD', 'symbol': 'BTC/USD', 'base': 'BTC', 'quote': 'USD',
                'precision': {'amount': 0.00000001, 'price': 0.01},
                'limits': {'amount': {'min': 0.00001, 'max': 1000}, 'price': {'min': 0.01, 'max': 1000000}, 'cost': {'min': 100}},
                'active': True, 'type': 'spot', 'spot': True, 'margin': False, 'contract': False
            }
            logger.info(f"[Executor] Registered BTC/USD to markets cache.")

if self.dry_run:
    # DRY RUN
    # CRIT-4 FIX: Apply slippage model so PnL accounting is realistic.
    # Market orders fill at mid-price + slippage. Without this, dry-run
    # win rates and daily loss caps are consistently optimistic.
    SLIPPAGE_PCT = 0.0002 # 0.02% == typical BTC/USDT USDM market order
    fill_price = (
        current_price * (1.0 + SLIPPAGE_PCT) if side == "buy"
        else current_price * (1.0 - SLIPPAGE_PCT)
    )
    cost = raw_size * fill_price
    logger.info(
        f"[DRY-RUN] {verdict} {ex_symbol} "
        f"| qty={raw_size:.6f} ({cost:.2f}) "
        f"| fill={fill_price:.2f} (slip={SLIPPAGE_PCT:.2%}) "
        f"| SL={stop_loss} TP={take_profit} "
        f"| equity=${equity:.2f} risk={max_risk_pct}% "
        f"| ULIS={ulis_verdict}"
    )
    doc_id = self._log_trade(ex_symbol, side, verdict, fill_price, stop_loss, take_profit, ulis_verdict)
    self.active_position = {
        "symbol": ex_symbol,
        "side": side,
        "size": raw_size,
        "entry_price": fill_price,
        "stop_loss": stop_loss,
        "take_profit": take_profit,
        "dry_run": True,
        "trade_doc_id": doc_id,
    }

# Telegram Notification
await self.notifier.send_trade_alert(
    symbol=ex_symbol, side=side, price=current_price,

```

[illegible]

[illegible]

```

        # This works correctly for both BUY and SELL positions due to how
        # Coinbase interprets the direction relative to the order side.
        stop_params["stop_direction"] = direction

        sl_order = await self.exchange.create_order(
            symbol=ex_symbol, type="limit", side=sl_side,
            amount=fmt_size,
            price=float(self.exchange.price_to_precision(ex_symbol, sl_limit)),
            params=stop_params,
        )
        sl_placed = True
        break
    except Exception as e:
        _sl_last_err = e
        logger.warning(f"[Executor] SL attempt {attempt+1}/3 failed: {e}")
        if attempt < 2:
            await asyncio.sleep(1.0)
    if not sl_placed:
        raise RuntimeError(f"SL placement failed after 3 attempts: {_sl_last_err}")
    sl_order_id = sl_order.get("id")
    logger.info(f"[Executor] SL attached at {stop_loss} (id={sl_order_id}) ✓")

    # Take-profit limit order
    tp_order = None
    _tp_last_err = None
    for attempt in range(3):
        try:
            tp_order = await self.exchange.create_order(
                symbol=ex_symbol,
                type="limit",
                side=sl_side,
                amount=fmt_size,
                price=float(self.exchange.price_to_precision(ex_symbol, take_profit)),
                params={"timeInForce": "GTC"},
            )
            tp_placed = True
            break
        except Exception as e:
            _tp_last_err = e
            logger.warning(f"[Executor] TP attempt {attempt+1}/3 failed: {e}")
            if attempt < 2:
                await asyncio.sleep(1.0)
    if not tp_placed:
        _tp_warn = (
            f"■■■ TP PLACEMENT FAILED for {side.upper()} {ex_symbol} @ {current_price}. "
            f"SL={stop_loss} IS active (id={sl_order_id}). "
            f"Position protected but NO take-profit. MONITOR MANUALLY."
        )
        logger.error(f"[Executor] {_tp_warn}")
        await self.notifier.send_error_alert(_tp_warn)
        tp_order_id = None
    else:
        tp_order_id = tp_order.get("id")
        logger.info(f"[Executor] TP attached at {take_profit} (id={tp_order_id}) ✓")

    # BUG-1 FIX: Use actual exchange fill price, not signal price.
    # Market orders fill at the ask (for buys) – using current_price
    # corrupts every downstream calculation (PnL, BE stop, daily limit).
    fill_price = float(order.get("average") or order.get("price") or current_price)
    logger.info(f"[Executor] Fill price: {fill_price:.2f} (signal was {current_price:.2f}, diff={fill_price-current_price:.2f})")

    doc_id = self._log_trade(ex_symbol, side, verdict, fill_price, stop_loss, take_profit, ulis_verdict)
    self.active_position = {
        "symbol": ex_symbol,
        "side": side,
        "size": fmt_size,
        "entry_price": fill_price,
        "stop_loss": stop_loss,
        "take_profit": take_profit,
        "order_id": order.get("id"),
        "sl_order_id": sl_order_id,
        "tp_order_id": tp_order_id,
        "sl_placed": sl_placed,
        "tp_placed": tp_placed,
        "dry_run": False,
        "trade_doc_id": doc_id,
    }
}

# Clear pending order since we now have an active position
self.pending_order = None

sl_tp_status = "SL+TP ✓" if (sl_placed and tp_placed) else ("SL ✓ | TP X MONITOR" if sl_placed else "■■■ NAKED")
logger.info(f"[Executor] Position open – protection status: {sl_tp_status}")
await self.notifier.send_trade_alert(
    symbol=ex_symbol, side=side, price=current_price,
    size=fmt_size, sl=stop_loss, tp=take_profit, is_dry=False
)
return True

except ccxt.InsufficientFunds as e:

```

```

        logger.error(f"[Executor] Insufficient funds: {e}")
    except ccxt.InvalidOrder as e:
        logger.error(f"[Executor] Invalid order: {e}")
    except Exception as e:
        err_msg = f"Live order / SL placement failed: {e}"
        logger.error(f"[Executor] {err_msg}", exc_info=True)

    # ■■■ EMERGENCY: SL failed – position is NAKED. Flatten immediately. ■■■
    # Note: TP-only failures are handled above and do NOT reach here –
    # the position keeps its SL in that case and is therefore still protected.
    if self.active_position is None and 'order' in locals() and order and order.get('id'):
        critical_msg = (
            f"■■■ CRITICAL: SL PLACEMENT FAILED after market fill! "
            f"FLATTENING NAKED {side.upper()} {ex_symbol} NOW. Error: {e}"
        )
        logger.critical(f"[Executor] {critical_msg}")
        await self.notifier.send_error_alert(critical_msg)

        close_side = "sell" if side == "buy" else "buy"
        try:
            await self.exchange.create_market_order(ex_symbol, close_side, fmt_size)
            logger.info(f"[Executor] ✓ Naked position flattened successfully.")
            await self.notifier.send_error_alert(
                f"■■■ Naked {side.upper()} {ex_symbol} position flattened. No unintended exposure remains."
            )
        except Exception as flatten_err:
            manual_msg = (
                f"■■■ CRITICAL FAILURE: Could not flatten naked {side.upper()} {ex_symbol} position! "
                f"MANUAL INTERVENTION REQUIRED IMMEDIATELY! Flatten error: {flatten_err}"
            )
            logger.critical(f"[Executor] {manual_msg}")
            await self.notifier.send_error_alert(manual_msg)
        else:
            await self.notifier.send_error_alert(err_msg)

    finally:
        # If we don't have an active position after all that,
        # we MUST clear pending_order so the bot isn't stuck "Waiting"
        if self.active_position is None:
            self.pending_order = None

# -----
# Position monitor (called from main loop for dry-run)
# -----

async def move_sl_to_breakeven(self, symbol: str, entry_price: float):

    if self._dry_run:
        logger.info(f"[Executor] DRY-RUN: Simulated moving SL to breakeven.")
        return

    logger.info(f"[Executor] Moving Stop Loss to Breakeven @ {entry_price:.2f}")
    try:
        if not self.active_position:
            return

        # Phase 1: Place the NEW Stop Loss FIRST
        # This ensures we NEVER have a naked position if the API fails
        fmt_size = self.exchange.amount_to_precision(symbol, self.active_position["size"])
        close_side = "sell" if self.active_position["side"] == "buy" else "buy"

        new_order = await self.exchange.create_order(
            symbol,
            "STOP_MARKET",
            close_side,
            fmt_size,
            None,
            params={
                "stopPrice": float(self.exchange.price_to_precision(symbol, entry_price)),
                "reduceOnly": True
            }
        )
        logger.info(f"[Executor] Phase 1 Success: New Breakeven SL safely placed on exchange.")

        # Phase 2: Now that new SL is secure, cancel the OLD Stop Loss
        open_orders = await self.exchange.fetch_open_orders(symbol)
        for order in open_orders:
            is_stop = order.get("type", "").lower() in ("stop", "stop_market", "stopmarket")
            is_not_new = str(order.get("id")) != str(new_order.get("id"))
            if is_stop and is_not_new:
                await self.exchange.cancel_order(order["id"], symbol)
                logger.debug(f"[Executor] Phase 2 Success: Cancelled old SL order {order['id']}")

        if hasattr(self, "notifier") and self.notifier:
            await self.notifier.send_message(f"■■■ **Breakeven Secured**\n"
            f"Moved Stop Loss to entry price at {entry_price:.2f} for {symbol}.")

    except Exception as e:
        # If Phase 1 fails, Phase 2 never runs. The original Stop Loss remains active.
        # Your capital is still fully protected.

```

[illegible]

[illegible]


```

        else:
            logger.warning(f"[Executor] Failed to cancel opposing order {cancel_id}: {e}")

def is_system_locked(self) -> bool:
    return time.time() < self.lock_expiry

async def engage_panic_mode(self, reason: str, lock_seconds: int = 300):
    logger.warning(f"[PanicMode] ENGAGED! Reason: {reason}. Locking system for {lock_seconds}s.")
    self.lock_expiry = time.time() + lock_seconds
    self.last_panic_reason = reason

    # Do NOT flatten positions – market orders into a hollowed order book
    # cause massive slippage. Let the hard Stop-Loss handle exits.

    # Notify user via Telegram
    if self.notifier:
        await self.notifier.send_message(f"■ PANIC MODE ENGAGED!\nReason: {reason}\nSystem locked for {lock_seconds}s.")

async def emergency_flatten(self, reason: str):
    if not self.active_position:
        return

    pos = self.active_position
    ex_symbol = pos["symbol"]
    side = pos["side"]
    size = pos["size"]
    close_side = "sell" if side == "buy" else "buy"

    logger.warning(f"[Executor] EMERGENCY FLATTEN triggered ({reason}) for {size} {ex_symbol}")

    try:
        if not self.dry_run:
            # 1. Cancel all open orders for this symbol first
            try:
                await self.exchange.cancel_all_orders(ex_symbol)
            except Exception:
                pass

            # 2. Market close
            await self.exchange.create_market_order(ex_symbol, close_side, size)

        logger.info(f"[Executor] Flattened {ex_symbol} ■")

        # BUG-2 + HIGH-1 FIX: Fetch actual fill price from trade history.
        # Old code: _last_price never set → fill_price always = entry → PnL always $0.
        # Every panic exit was recorded as flat, bypassing the daily loss limit.
        entry = pos.get("entry_price", 0.0)
        size = pos.get("size", 0.0)
        side = pos.get("side", "buy")
        fill_price = entry # fallback
        est_pnl = 0.0
        if not self.dry_run:
            try:
                recent = await self.exchange.fetch_my_trades(ex_symbol, limit=5)
                closing = [
                    t for t in recent
                    if float(t.get("info", {}).get("realizedPnl", 0)) != 0
                ]
                if closing:
                    fill_price = float(closing[-1]["price"])
                    est_pnl = float(closing[-1].get("info", {}).get("realizedPnl", 0))
                    logger.info(f"[Flatten] Exchange PnL: ${est_pnl:.2f} fill={fill_price:.2f}")
                else:
                    # Estimate from entry vs current candle close
                    raw = (fill_price - entry) * size if side == "buy" else (entry - fill_price) * size
                    est_pnl = raw - (entry + fill_price) * size * self.TAKER_FEE
            except Exception as fe:
                logger.warning(f"[Flatten] Could not fetch fill: {fe}")

        # HIGH-1 FIX: pass real fill_price (not 0.0) to Firestore
        self._last_panic_pnl = est_pnl # BUG-3: expose for main.py daily_pnl update
        self._update_trade_exit(pos.get("trade_doc_id"), fill_price, est_pnl)
        self.active_position = None
    except Exception as e:
        # 3.4 FIX: Critical failure requires immediate human action.
        # Fire Telegram and log at CRITICAL level regardless of any other state.
        msg = (
            f"■ CRITICAL: FAILED TO FLATTEN POSITION\n"
            f"Symbol: {ex_symbol} | Side: {side.upper()} | Size: {size}\n"
            f"Reason: {reason}\n"
            f"Error: {str(e)}\n"
            f"ACTION REQUIRED: Log into exchange immediately and close this position manually."
        )
        logger.critical(msg)
        if self.notifier:
            try:
                await self.notifier.send_message(msg)
            except Exception as notify_err:

```

```
        logger.error(f"[Executor] Also failed to send Telegram alert: {notify_err}")
# P0-3 FIX: Use a dedicated boolean flag, NOT a sentinel dict.
# The 30s heartbeat poll calls _check_live_position_exit which sets
# active_position = None if the symbol is gone – silently wiping the
# sentinel dict and allowing the bot to resume trading unprotected.
# A separate boolean survives the poll clearing active_position.
self._flatten_failed = True
self.active_position = None # Clear position – halt is enforced via _flatten_failed flag
```

Signal Configuration & Parameters

```

# Bayesian / Confidence
MIN_BAYESIAN = 0.62 # Minimum posterior confidence for any entry
MIN_BAYESIAN_STRONG = 0.72 # Strong-signal threshold (STRONG_LONG / STRONG_SHORT)

# ■■■ Z-Score (Student's t-distribution VWAP Z) ■■■
# M-02 FIX: _vwap_z_score_t() already shrinks the Gaussian Z by t_scale=0.8165.
# Thresholds must be multiplied by 0.8165 to restore the intended σ boundaries.
Z_OVERSOLD = -1.22 # Student-t adj: was -1.5
Z_OVERBOUGHT = 1.22 # Student-t adj: was 1.5
MEAN_REV_Z_THRESHOLD = 1.22 # Student-t adj: was 1.5 – matches NEUTRAL z_threshold

# ■■■ OFI (tanh output, range -1 to +1) ■■■
OFI_STRONG_THRESHOLD = 0.6 # Strong directional flow (equivalent to old |OFI| > 40)
OFI_MID_THRESHOLD = 0.3 # Moderate flow (equivalent to old |OFI| > 10)
OFI_WEAK_THRESHOLD = 0.15 # Weak / noise floor

# ■■■ Skewness ■■■
SKEW_SIGNIFICANT = 0.3 # |skewness| above this is considered meaningful

# ■■■ Risk / Position Sizing ■■■
MAX_RISK_PCT = 1.0 # Max risk per trade as % of equity
MAX_DAILY_LOSS_PCT = 3.0 # Stop trading if daily loss hits this %
MAX_DRAWDOWN_PCT = 15.0 # Hard session drawdown halt (restart required)
ATR_SL_MULTIPLIER = 1.5 # SL = entry ± ATR × this
ATR_TP_MULTIPLIER = 3.0 # TP = entry ± ATR × this (3:2 RR minimum)

# ■■■ Strategy Scoring ■■■
TREND_SCORE_MIN = 1.5 # Minimum composite score for a trend entry
WALL_PROXIMITY_PCT = 0.003 # % distance to qualify as LIQUIDITY regime

# ■■■ ULIS Gate ■■■
ULIS_CONFIDENCE_STRONG = 0.68 # ALDE confidence threshold for STRONG_LONG/SHORT
ULIS_CASCADE_ABORT = 0.65 # cascade_risk above this → AVOID verdict

# ■■■ Cooldowns ■■■
POST_TRADE_COOLDOWN_S = 90 # seconds after any exit before new entry allowed
CASCADE_COOLDOWN_S = 300 # seconds after SL exit (cascade prevention)
CONSECUTIVE_LOSS_HALT = 3 # was 2 – reduces false lockouts

COLD_START_TRADE_COUNT = 30
COLD_START_CONFIDENCE_DISCOUNT = 0.05

# ■■■ REGIME-CONDITIONAL PARAMETER MATRIX (HMM Spec, Apr 2026) ■■■
# ■■■
# Replaces fixed thresholds (e.g. z >= 2.2 always) with adaptive ones that fit
# the current market environment as classified by the HMM.
#
# Regime → Strategy mapping:
# RANGE → Mean-reversion dominant (quiet market, tight SL, low Z bar)
# NEUTRAL → All strategies active (default / transitional)
# TREND → Trend-following dominant (noisy market, wide SL, high Z bar)
# LIQUIDITY → Wall-proximity sweep (same defaults as NEUTRAL, no HTF block)
#
REGIME_PARAMS = {
    "RANGE": {
        # Low-volatility: small deviations are statistically meaningful
        "z_threshold": 1.06, # M-02 FIX: was 1.3, Student-t adj (1.3×0.8165)
        "atr_multiplier_sl": 1.14, # Was 1.2 – 5% reduction post-Wilder compensation
        "ofi_bound": 0.08, # Was 0.10. Needs less order flow to enter
        "min_confidence": 0.62, # Was 0.55 – raised to match LIQUIDITY threshold
        "rr_target": 1.8, # 1.8:1 minimum R:R
        "be_lock_trigger": 0.8, # Move SL to break-even after 0.8×ATR profit
        "panic_threshold": 2.5, # Flash-crash trigger
        "candle_gate_sec": 30, # Was 45s. Faster entry
        "htf_block": False, # Counter-HTF is the strategy in RANGE
        "cascade_cooldown_s": 180, # PHASE-2.5: was 300s, lowered to 180s for RANGE recovery
    },
    "NEUTRAL": {
        # Normal-volatility: balanced thresholds
        "z_threshold": 1.22, # M-02 FIX: was 1.5, Student-t adj (1.5×0.8165)
        "atr_multiplier_sl": 1.43,
        "ofi_bound": 0.10, # PHASE-4.1: was 0.12, lowered to 0.10 for more signal pass-through
        "min_confidence": 0.62, # was 0.55 – raised to match LIQUIDITY threshold
        "rr_target": 2.0,
        "be_lock_trigger": 1.5, # widened: move SL to BE only after 1.5×ATR profit
        "panic_threshold": 5.0,
        "candle_gate_sec": 45, # Was 60s
        "htf_block": True,
        "cascade_cooldown_s": 300, # STRATEGY-B: 5min (default)
    },
    "TREND": {
        # High-volatility: trend-following
        "z_threshold": 2.0, # Was 2.5. Triggers on shallower pullbacks
        "atr_multiplier_sl": 2.09, # Was 2.2 – 5% reduction post-Wilder compensation
    }
}

```

```

    "ofi_bound": 0.20, # Was 0.25
    "min_confidence": 0.65, # Was 0.72. Massive increase in trend trades
    "rr_target": 2.5,
    "be_lock_trigger": 2.0, # widened: let trend breathe – do not lock BE until 2×ATR profit
    "panic_threshold": 8.0,
    "candle_gate_sec": 60, # Was 90s
    "htf_block": True,
    "cascade_cooldown_s": 240, # STRATEGY-B: 4min (trend may still be valid)
  },
  "LIQUIDITY": {
    # Wall-proximity sweeps
    "z_threshold": 1.22, # M-02 FIX: was 1.5, Student-t adj (1.5×0.8165)
    "atr_multiplier_sl": 1.43,
    "ofi_bound": 0.12,
    "min_confidence": 0.62, # FIX-P10: was 0.55 which == sweep neutralizer floor (no-op); raised to 0.62 so threshold i
    "rr_target": 2.0,
    "be_lock_trigger": 1.5, # widened: move SL to BE only after 1.5×ATR profit
    "panic_threshold": 5.0,
    "candle_gate_sec": 45, # Was 60s
    "htf_block": False,
    "cascade_cooldown_s": 180, # STRATEGY-B: 3min (sweep may repeat next candle)
  },
}

```

ULIS Strategy Engine

```
# Sweep / BOS Counters (derived from candle history)
```

```

        return sweep_count, bos_list

# P1-4 FIX: Filter to significant walls only (size > median * 5x) before computing sweeps.
bid_prices = sorted(bids_dict.keys(), reverse=True)
ask_prices = sorted(asks_dict.keys())
if not bid_prices or not ask_prices:
    return sweep_count, bos_list

# Compute median sizes to identify significant walls
bid_sizes = [bids_dict[p] for p in bid_prices]
ask_sizes = [asks_dict[p] for p in ask_prices]
import statistics
median_bid = statistics.median(bid_sizes) if bid_sizes else 0
median_ask = statistics.median(ask_sizes) if ask_sizes else 0
wall_threshold_bid = median_bid * 5 if median_bid > 0 else 0
wall_threshold_ask = median_ask * 5 if median_ask > 0 else 0

# Filter to significant walls only
significant_bids = {p: q for p, q in bids_dict.items() if q >= wall_threshold_bid}
significant_asks = {p: q for p, q in asks_dict.items() if q >= wall_threshold_ask}

sig_bid_prices = sorted(significant_bids.keys(), reverse=True)
sig_ask_prices = sorted(significant_asks.keys())
if not sig_bid_prices or not sig_ask_prices:
    return sweep_count, bos_list

nearest_bid = sig_bid_prices[0]
nearest_ask = sig_ask_prices[0]

recent = list(candles)[-10:]
for i in range(1, len(recent)):
    c = recent[i]
    p = recent[i - 1]
    if p["high"] > nearest_ask and c["close"] < nearest_ask:
        sweep_count += 1
    if p["low"] < nearest_bid and c["close"] > nearest_bid:
        sweep_count += 1
    # BOS: consecutive higher-lows or lower-highs = structure break
    if c["close"] > p["high"]:
        bos_list.append({"dir": "up", "price": c["close"]})
    elif c["close"] < p["low"]:
        bos_list.append({"dir": "down", "price": c["close"]})

return sweep_count, bos_list

# =====
# FVG Counter (Fair Value Gap) from candles
# =====

def _count_fvgs(candles: list) -> int:
    count = 0
    recent = list(candles)[-15:]
    for i in range(2, len(recent)):
        prev2 = recent[i - 2]
        curr = recent[i]
        if curr["low"] > prev2["high"]: # bullish FVG
            count += 1
        elif curr["high"] < prev2["low"]: # bearish FVG
            count += 1
    return count

# =====
# Order Book Helpers
# =====

def _book_to_levels(book_dict: Dict[float, float]) -> List[Dict[str, float]]:
    return [{"price": p, "size": s, "classification": "WALL" if s > 0 else "NORMAL"}
            for p, s in book_dict.items()]

# =====
# Main ULIS Verdict Function
# =====

MIN_SWEEP_CONFIRMS = 2

def _sweep_micro_confirms(metrics: dict, sweep_direction: str) -> int:
    confirms = 0
    rsi = metrics.get("rsi", 50.0)
    if sweep_direction == "bull" and rsi < 48:
        confirms += 1
    elif sweep_direction == "bear" and rsi > 52:
        confirms += 1
    ofi = metrics.get("ofi", 0.0)
    if sweep_direction == "bull" and ofi > 0.15:
        confirms += 1
    elif sweep_direction == "bear" and ofi < -0.15:
        confirms += 1

```

[illegible]

```
liquidity_vector = _clamp(ulp - dlp, -1.0, 1.0)

# ■ Phase 5: ALDE Confidence Score ████████████████████████████████████████████████████
ilih_score = _normalize(total_depth, 1000.0, 80000.0)
alde_conf0 = (
    0.25 * ilih_score +
    0.22 * scores["glrScore"] +
    0.20 * scores["nlfScore"] + # reduced from 0.25: bayes already priced via nlf_score
    0.20 * (1.0 - scores["reflexivityScore"]) +
    0.13 * (1.0 - scores["fragility"])
)
# 3.2 FIX: Removed bayes_boost – bayesianPosterior already feeds nlf_score (double-counting).
# Only CVD boost remains as it's an orthogonal (independent) signal.
cvd_boost = math.tanh(cvd / 50_000.0) * 0.06 * math.copysign(1, liquidity_vector or 1)
alde_confidence = _clamp(alde_conf0 + cvd_boost, 0.0, 1.0)

# ■ Phase 6: Verdict AND-Gates ████████████████████████████████████████████████████
is_high_volatility = scores["fragility"] > 0.55 or scores["reflexivityScore"] > 0.55
is_mean_reversion = abs(z_score) > 1.8 and not trend_agreement
is_liquidity_vacuum = scores["visibleLiquidity"] < 0.35 and fvg_count > 1
is_bos_confirmed = len(bos_list) > 0
cascade_warning = cascade_risk > 0.60

if cascade_risk > 0.65 and scores["reflexivityScore"] > 0.50:
    verdict = "AVOID"
    regime_label = "Cascade Risk Critical – Liquidity Singularity · STAND ASIDE"
elif cascade_warning and is_high_volatility and cascade_risk > 0.55:
    verdict = "UNWIND"
    regime_label = "Pre-Cascade: Volatility Expansion · Reduce Exposure Now"
elif market_state == "UNSTABLE" and not is_mean_reversion:
    verdict = "NEUTRAL"
    regime_label = "Market Unstable – Cascade Elevated · Wait for Stabilisation"
elif (
    liquidity_vector > 0.28 and
    scores["nlfScore"] > 0.58 and
    scores["glrScore"] > 0.50 and
    cascade_risk < 0.45 and
    alde_confidence > 0.68 and
    bayes > 0.58
):
    verdict = "STRONG_LONG"
    regime_label = "ALDE+ULIS Bullish Confluence · Full Alignment – High Probability Long"
elif liquidity_vector > 0.08 and alde_confidence > 0.52 and cascade_risk < 0.62:
    verdict = "LONG"
    regime_label = "Bullish Bias · Liquidity Vector Positive – Moderate ALDE Alignment"
elif (
    liquidity_vector < -0.28 and
    scores["nlfScore"] < 0.42 and
    scores["glrScore"] < 0.50 and
    cascade_risk < 0.45 and
    alde_confidence > 0.68 and
    bayes < 0.42
):
    verdict = "STRONG_SHORT"
    regime_label = "ALDE+ULIS Bearish Confluence · Full Alignment – High Probability Short"
elif liquidity_vector < -0.08 and alde_confidence > 0.52 and cascade_risk < 0.62:
    verdict = "SHORT"
    regime_label = "Bearish Bias · Liquidity Vector Negative – Moderate ALDE Alignment"
elif is_mean_reversion:
    verdict = "SHORT" if z_score > 0 else "LONG"
    regime_label = "Mean Reversion Play – Z-Score Extreme · Fade the Extension"
elif is_liquidity_vacuum and abs(liquidity_vector) > 0.12:
    verdict = "BREAKOUT_WATCH"
    regime_label = "Liquidity Vacuum · Breakout Conditions Forming"
else:
    verdict = "NEUTRAL"
    regime_label = "ALDE Equilibrium – Liquidity Vector Near Zero · No Actionable Edge"

# ■ Determine trade gate ████████████████████████████████████████████████████
should_trade = verdict not in ("AVOID", "UNWIND")

# ■ Confidence modifier ████████████████████████████████████████████████████
confidence_boost = 0.0
if verdict == "STRONG_LONG" or verdict == "STRONG_SHORT":
    confidence_boost = 0.08
elif verdict == "AVOID" or verdict == "UNWIND":
    confidence_boost = -1.0 # This ensures confidence drops below threshold
elif verdict == "NEUTRAL":
    confidence_boost = 0.0 # FIX-P3: NEUTRAL = no actionable edge → zero boost (was +0.03 which inflated every entry)
elif verdict == "BREAKOUT_WATCH":
    confidence_boost = -0.02 # Mild caution: breakout vacuum, not a directional veto

logger.info(
    f"[ULIS] verdict={verdict} | vector={liquidity_vector:.3f} | "
    f"cascade={cascade_risk:.2f} | conf={alde_confidence:.2%} | "
    f"state={market_state} | sweeps={sweep_count} | BOS={len(bos_list)}"
)

return {
    "verdict": verdict,

```



```
"confidence":      round(alde_confidence, 4),
"cascade_risk":    round(cascade_risk, 4),
"liquidity_vector": round(liquidity_vector, 4),
"regime_label":    regime_label,
"should_trade":    should_trade,
"confidence_boost": confidence_boost,
"ulp":             round(ulp, 4),
"dlp":             round(dlp, 4),
"market_state":    market_state,
"sweep_count":     sweep_count,
}
```

■ data_feed.py

Market Data Handler

```
import asyncio
import json
import logging
import time
import websockets
import httpx
from collections import deque

logger = logging.getLogger(__name__)

class MarketState:
    def __init__(self, symbol: str):
        self.symbol = symbol.upper()
        self.candles: deque = deque(maxlen=200)
        # Keep all trades received; prune old ones in add_trade
        self.recent_trades: deque = deque(maxlen=50000)
        # Order Book snapshot { price_float: size_float }
        self.bids: dict = {}
        self.asks: dict = {}
        # Running cumulative volume delta
        self.cvd: float = 0.0
        # Binance USDM funding rate - fetched periodically via REST.
        # Positive = longs pay shorts (crowded long → bearish pressure).
        # Negative = shorts pay longs (crowded short → bullish squeeze).
        self.funding_rate: float = 0.0
        self.basis: float = 0.0 # Futures mark price - Spot close
        self.mark_price: float = 0.0 # Latest Futures mark price
        self._reconnect_event: asyncio.Event = asyncio.Event()
        self.candle_close_event: asyncio.Event = asyncio.Event() # NEW: Event-driven execution trigger
        self._last_trade_ts: float = time.time() # epoch-seconds of last aggTrade received (P0-1 FIX: was 0.0 → time.time())
        self._cvd_was_reset: bool = False # flag to suppress CVD delta spike after reconnect
        self._aggtrade_msg_count: int = 0 # P0-1 FIX: throughput counter for monitoring
        self._aggtrade_count_reset_ts: float = time.time() # last reset for msg/min calculation

# -----
# Candle management
# -----
def add_candle(self, c_data: dict, is_final: bool):

    candle = {
        'time': c_data['t'] / 1000,
        'open': float(c_data['o']),
        'high': float(c_data['h']),
        'low': float(c_data['l']),
        'close': float(c_data['c']),
        'volume': float(c_data['v'])
    }
    if is_final:
        # Closed candle - always append
        self.candles.append(candle)
        self.candle_close_event.set() # Trigger zero-latency execution
    else:
        # Live candle - replace if same timestamp, otherwise append
        if self.candles and self.candles[-1]['time'] == candle['time']:
            self.candles[-1] = candle
        else:
            self.candles.append(candle)

# -----
# Trade tape
# -----
def add_trade(self, t_data: dict):
    price = float(t_data['p'])
    size = float(t_data['q'])
    is_buyer_maker = t_data['m']
    # If buyer is maker they were resting → the *seller* aggressed → SELL tape
    side = 'SELL' if is_buyer_maker else 'BUY'

    trade = {
        'price': price,
        'size': size,
        'usd_volume': price * size,
        'side': side,
        'time': t_data['T'] # Binance millisecond timestamp
    }
    self._last_trade_ts = t_data['T'] / 1000.0
    self._aggtrade_msg_count += 1
    self.recent_trades.append(trade)

    # Update CVD
    delta = size if side == 'BUY' else -size
    self.cvd += delta

    # Prune trades older than 5 minutes using wall-clock time
    now_ms = time.time() * 1000
    while self.recent_trades and (now_ms - self.recent_trades[0]['time']) > 300_000:
```

```

        self.recent_trades.popleft()

# -----
# Order book
# -----
def update_depth(self, depth_data: dict):

    raw_b = depth_data.get('b', depth_data.get('bids', []))
    raw_a = depth_data.get('a', depth_data.get('asks', []))
    self.bids = {float(p): float(q) for p, q in raw_b}
    self.asks = {float(p): float(q) for p, q in raw_a}
    # Remove levels with zero quantity (Binance sends these as deletes)
    self.bids = {p: q for p, q in self.bids.items() if q > 0}
    self.asks = {p: q for p, q in self.asks.items() if q > 0}

class BinanceDataFeed:

    def __init__(self, symbol: str = "BTCUSDT", interval: str = "15m", testnet: bool = True):
        self.symbol = symbol.lower()
        self.interval = interval.lower()
        self.state = MarketState(symbol)

        # Binance USDM Futures endpoints (separate from Spot)
        if testnet:
            self.rest_url = "https://testnet.binancefuture.com"
            base_url = "wss://stream.binancefuture.com"
        else:
            # LIVE FUTURES DATA BLOCKED IN EU/US – using Spot Proxy for WS Data
            self.rest_url = "https://fapi.binance.com" # REST works via API keys/CDN
            base_url = "wss://stream.binance.com:9443" # Spot WebSocket bypasses geoblock

        streams = (
            f"{self.symbol}@kline_{self.interval}"
            f"/{self.symbol}@aggTrade" # Spot & Futures both use aggTrade
            f"/{self.symbol}@depth20@100ms"
        )
        self.ws_url = f"{base_url}/stream?streams={streams}"
        self.is_running = False
        self._rest_fetch_lock = asyncio.Lock()
        self._last_funding_fetch: float = 0.0 # epoch-seconds of last funding rate REST call
        self._last_kline_frame_ts: float = 0.0 # epoch-seconds of last kline WS frame received

# -----
# Message routing
# -----
async def _handle_message(self, raw: str):
    try:
        payload = json.loads(raw)
    except json.JSONDecodeError as e:
        logger.warning(f"JSON decode error: {e}")
        return

    if 'stream' not in payload:
        return # Subscription confirmations etc.

    stream: str = payload['stream']
    data: dict = payload['data']

    try:
        if '@kline_' in stream:
            self.state.add_candle(data['k'], data['k']['x'])
            self._last_kline_frame_ts = time.time() # P0-2: track freshness for health monitor
        elif '@aggTrade' in stream:
            # Futures aggTrade uses same fields as Spot trade (p, q, m, T)
            self.state.add_trade(data)
        elif '@depth' in stream:
            self.state.update_depth(data)
    except KeyError as e:
        logger.error(f"Missing key in {stream} payload: {e}")
    except Exception as e:
        logger.error(f"Error processing stream '{stream}': {e}", exc_info=True)

# -----
# Funding Rate Fetch (periodic, every 60 s)
# -----
async def _fetch_funding_rate(self):
    url = f"{self.rest_url}/fapi/v1/premiumIndex"
    params = {"symbol": self.symbol.upper()}
    import os
    api_key = os.environ.get("BINANCE_API_KEY", "")
    headers = {"X-MBX-APIKEY": api_key} if api_key else {}
    try:
        async with httpx.AsyncClient(timeout=6.0) as client:
            resp = await client.get(url, params=params, headers=headers)
            resp.raise_for_status()
            data = resp.json()
            rate = float(data.get("lastFundingRate", 0.0))
            self.state.funding_rate = rate

```

```

# NEW: Track Futures-Spot basis for SL/TP price correction
mark_price = float(data.get("markPrice", 0.0))
index_price = float(data.get("indexPrice", 0.0))
if mark_price > 0 and len(self.state.candles) > 0:
    spot_close = self.state.candles[-1]["close"]
    self.state.basis = mark_price - spot_close # positive = Futures premium
    self.state.mark_price = mark_price
else:
    self.state.basis = 0.0
    self.state.mark_price = mark_price
logger.info(
    f"[DataFeed] Funding rate: {rate:+.6f} ({rate*100:+.4f}%) | "
    f"Basis: {getattr(self.state, 'basis', 0.0):+.2f}"
)
except Exception as e:
    logger.warning(f"[DataFeed] Failed to fetch funding rate: {e}")

async def funding_rate_loop(self):
    await asyncio.sleep(5)
    while self.is_running:
        await self._fetch_funding_rate()
        try:
            await asyncio.sleep(60)
        except asyncio.CancelledError:
            break

# -----
# REST API Prefetch
# -----
async def _fetch_historical_candles_rest(self):
    async with self._rest_fetch_lock:
        # Futures klines live under /fapi/v1/klines, NOT /api/v3/klines (spot)
        url = f"{self.rest_url}/fapi/v1/klines"
        params = {
            "symbol": self.symbol.upper(),
            "interval": self.interval,
            "limit": 100
        }
        import os
        api_key = os.environ.get("BINANCE_API_KEY", "")
        headers = {"X-MBX-APIKEY": api_key} if api_key else {}

        # 3.3 FIX: Snapshot CVD before any mutation.
        # If REST fails (network drop, reconnect), we preserve the existing baseline
        # rather than resetting to 0.0 which would produce a false CVD delta spike.
        prev_cvd = self.state.cvd

        try:
            logger.info(f"[DataFeed] Fetching historical {self.interval} candles from {url}...")
            async with httpx.AsyncClient(timeout=10.0) as client:
                resp = await client.get(url, params=params, headers=headers)
                resp.raise_for_status()
                data = resp.json()

            # Reset CVD to re-anchor perfectly based on REST history (only on success)
            self.state.cvd = 0.0
            parsed_candles = []
            for k in data:
                # Binance REST returns an array of arrays
                c_data = {
                    't': int(k[0]),
                    'o': float(k[1]),
                    'h': float(k[2]),
                    'l': float(k[3]),
                    'c': float(k[4]),
                    'v': float(k[5]),
                }
                parsed_candles.append({"close": float(k[4]), "high": float(k[2]), "low": float(k[3]), "volume": float(k[5])})
                self.state.add_candle(c_data, is_final=True)

            # Rebuild CVD analytically
            taker_buy_base = float(k[9])
            vol = float(k[5])
            self.state.cvd += (2.0 * taker_buy_base) - vol

            logger.info(f"[DataFeed] Successfully loaded {len(self.state.candles)} historical candles. CVD rebuilt: {self.state.cvd}")
            self.candles_seeded = True
            return parsed_candles
        except Exception as e:
            logger.warning(
                f"[DataFeed] Failed to prefetch historical candles ({e}). "
                f"Preserving existing CVD={prev_cvd:.0f} to avoid false delta spike."
            )
            self.state.cvd = prev_cvd # restore - do not corrupt the signal

# -----
# Connection loop with exponential back-off
# -----

```

```

async def run(self):
    self.is_running = True
    retry_delay = 1

    if not getattr(self, "candles_seeded", False):
        await self._fetch_historical_candles_rest()

    while self.is_running:
        try:
            logger.info(f"[DataFeed] Connecting → {self.ws_url}")
            async with websockets.connect(
                self.ws_url,
                ping_interval=20,
                ping_timeout=30,
                close_timeout=10
            ) as ws:
                retry_delay = 1 # Reset back-off on successful connect
                self.state.cvd = 0.0
                self.state._cvd_was_reset = True # Signal main loop to suppress next delta
                logger.info("[DataFeed] Connected ✓")
                # PHASE-0.3: Fetch funding rate immediately on connection (before WS loop)
                asyncio.create_task(self._fetch_funding_rate())
                import asyncio as _asyncio
                while self.is_running and not self.state._reconnect_event.is_set():
                    try:
                        msg = await _asyncio.wait_for(ws.recv(), timeout=120)
                        await self._handle_message(msg)
                    except _asyncio.TimeoutError:
                        logger.warning("[DataFeed] recv() timeout (120s) – connection may be frozen. Reconnecting...")
                        break
                    except websockets.exceptions.ConnectionClosedOK:
                        break
                # Clear reconnect signal so the outer loop reconnects immediately
                if self.state._reconnect_event.is_set():
                    self.state._reconnect_event.clear()
                logger.info('[DataFeed] Reconnect event consumed - reconnecting now.')

            except websockets.exceptions.ConnectionClosedOK:
                logger.info("[DataFeed] Connection closed cleanly.")
            except websockets.exceptions.ConnectionClosed as e:
                logger.warning(f"[DataFeed] Connection dropped: {e}. Reconnecting in {retry_delay}s...")
            except OSError as e:
                logger.error(f"[DataFeed] Network error: {e}. Retrying in {retry_delay}s...")
            except Exception as e:
                logger.error(f"[DataFeed] Unexpected error: {e}", exc_info=True)

        if self.is_running:
            await asyncio.sleep(retry_delay)
            retry_delay = min(retry_delay * 2, 60)

    async def feed_health_monitor(self, notifier=None):

        interval_secs = {
            "1m": 60, "3m": 180, "5m": 300, "15m": 900, "1h": 3600
        }.get(self.interval, 900)
        stale_threshold = interval_secs * 3.0

        # Give the feed 60s to warm up before monitoring
        await asyncio.sleep(60)

        # FIX: Track boot time so we can detect "kline never arrived at all"
        _monitor_boot_ts = time.time()

        while self.is_running:
            await asyncio.sleep(60)
            if not self.is_running:
                break

            # FIX: If kline frames have NEVER arrived (e.g. geoblock), force
            # reconnect after 180s instead of skipping forever. The old code
            # did `continue` here, meaning a fully dead kline stream would
            # never trigger a reconnect.
            if self._last_kline_frame_ts == 0.0:
                secs_since_boot = time.time() - _monitor_boot_ts
                if secs_since_boot > 180:
                    alert_msg = (
                        f"■■■ [DataFeed] FEED DEAD: zero kline frames received "
                        f"since boot ({secs_since_boot:.0f}s ago). Forcing reconnect."
                    )
                    logger.error(alert_msg)
                    if notifier:
                        try:
                            await notifier.send_message(alert_msg)
                        except Exception:
                            pass
                    self.state._reconnect_event.set()
                    _monitor_boot_ts = time.time() # reset to avoid spam
                    continue

            age = time.time() - self._last_kline_frame_ts

```

[illegible]

```

        params={"listenKey": listen_key}
    )
    logger.info("[UserDataStream] listenKey keepalive ✓")
except Exception as e:
    logger.warning(f"[UserDataStream] Keepalive failed: {e}")

keepalive_task = asyncio.create_task(_keepalive())

async with websockets.connect(
    f"{ws_base}/ws/{listen_key}",
    ping_interval=20, ping_timeout=30
) as ws:
    logger.info("[UserDataStream] Connected ✓")
    while self.is_running:
        try:
            raw = await asyncio.wait_for(ws.recv(), timeout=60)
            event = json.loads(raw)
            if event.get("e") == "ORDER_TRADE_UPDATE":
                order = event.get("o", {})
                # Filled + reduceOnly = position exit
                if order.get("X") == "FILLED" and order.get("R"):
                    pnl = float(order.get("rp", 0.0))
                    logger.info(
                        f"[UserDataStream] Fill: orderId={order.get('i')} "
                        f"PnL=${pnl:.2f}"
                    )
                    await on_fill_callback(pnl)
        except asyncio.TimeoutError:
            logger.warning("[UserDataStream] recv timeout – reconnecting")
            break

    keepalive_task.cancel()

except Exception as e:
    logger.warning(f"[UserDataStream] Error: {e} – retry in 10s")
    await asyncio.sleep(10)

```

■ derivatives_context.py

Derivatives Context Manager

```
import httpx
import asyncio
import logging
import time

logger = logging.getLogger(__name__)

class DerivativesContext:

    def __init__(self, symbol: str = "BTCUSDT"):
        self.symbol = symbol
        self._cache: dict = {}
        self._cache_ts: dict = {}
        self.CACHE_TTL = 300 # 5 minutes

    async def _fetch(self, key: str, url: str, params: dict = None) -> dict:
        now = time.time()
        if key in self._cache and (now - self._cache_ts.get(key, 0)) < self.CACHE_TTL:
            return self._cache[key]
        try:
            async with httpx.AsyncClient(timeout=8.0) as client:
                resp = await client.get(url, params=params)
                resp.raise_for_status()
                data = resp.json()
                self._cache[key] = data
                self._cache_ts[key] = now
            return data
        except Exception as e:
            logger.warning(f"[Derivatives] {key} fetch failed: {e}")
            return {}

    async def get_open_interest(self) -> dict:
        current = await self._fetch(
            "oi_current",
            "https://fapi.binance.com/fapi/v1/openInterest",
            {"symbol": self.symbol}
        )
        history = await self._fetch(
            "oi_history",
            "https://fapi.binance.com/futures/data/openInterestHist",
            {"symbol": self.symbol, "period": "15m", "limit": 30}
        )

        current_oi = float(current.get("openInterest", 0))

        oi_change_pct = 0.0
        oi_momentum_1h = 0.0

        if history and len(history) >= 4:
            prev_oi = float(history[-2].get("sumOpenInterest", current_oi))
            oi_change_pct = (current_oi - prev_oi) / max(prev_oi, 1) * 100
            oi_1h_ago = float(history[-4].get("sumOpenInterest", current_oi))
            oi_momentum_1h = (current_oi - oi_1h_ago) / max(oi_1h_ago, 1) * 100

        return {
            "current_oi": current_oi,
            "oi_change_pct_15m": oi_change_pct,
            "oi_momentum_1h": oi_momentum_1h,
            "oi_collapsing": oi_momentum_1h < -2.0,
        }

    async def get_top_trader_positioning(self) -> dict:
        data = await self._fetch(
            "top_trader_ls",
            "https://fapi.binance.com/futures/data/topLongShortPositionRatio",
            {"symbol": self.symbol, "period": "15m", "limit": 10}
        )
        if not data:
            return {"top_long_pct": 50.0, "crowd_signal": "NEUTRAL"}

        latest = data[-1]
        long_pct = float(latest.get("longAccount", 0.5)) * 100
        short_pct = 100 - long_pct

        if long_pct > 70:
            crowd_signal = "CROWDED_LONG" # contrarian bearish
        elif short_pct > 70:
            crowd_signal = "CROWDED_SHORT" # contrarian bullish squeeze risk
        else:
            crowd_signal = "NEUTRAL"

        return {
            "top_long_pct": long_pct,
            "top_short_pct": short_pct,
            "crowd_signal": crowd_signal
        }
```



```

        "top_short_pct": short_pct,
        "crowd_signal": crowd_signal,
    }
}

async def get_options_context(self) -> dict:
    try:
        data = await self._fetch(
            "deribit_options",
            "https://www.deribit.com/api/v2/public/get_book_summary_by_currency",
            {"currency": "BTC", "kind": "option"}
        )
        instruments = data.get("result", [])
        if not instruments:
            return {"put_call_ratio": 1.0, "options_signal": "NEUTRAL"}

        total_put_oi = sum(
            float(i.get("open_interest", 0))
            for i in instruments if "-P" in i.get("instrument_name", "")
        )
        total_call_oi = sum(
            float(i.get("open_interest", 0))
            for i in instruments if "-C" in i.get("instrument_name", "")
        )

        put_call_ratio = total_put_oi / max(total_call_oi, 1)

        if put_call_ratio > 1.2:
            options_signal = "FEAR"          # institutions buying downside protection
        elif put_call_ratio < 0.8:
            options_signal = "GREED"         # no hedging = complacent = contrarian caution
        else:
            options_signal = "NEUTRAL"

        return {
            "put_call_ratio": round(put_call_ratio, 3),
            "options_signal": options_signal,
        }
    except Exception as e:
        return {"put_call_ratio": 1.0, "options_signal": "NEUTRAL", "error": str(e)}

async def get_taker_flow(self) -> dict:
    data = await self._fetch(
        "taker_flow",
        "https://fapi.binance.com/futures/data/takerlongshortRatio",
        {"symbol": self.symbol, "period": "5m", "limit": 12}
    )
    if not data:
        return {"taker_imbalance": 0.0}

    latest = data[-1]
    buy_vol = float(latest.get("buyVol", 0))
    sell_vol = float(latest.get("sellVol", 0))
    total = buy_vol + sell_vol

    return {
        "taker_buy_ratio": buy_vol / max(total, 1),
        "taker_sell_ratio": sell_vol / max(total, 1),
        "taker_imbalance": (buy_vol - sell_vol) / max(total, 1),
    }

async def get_full_context(self) -> dict:
    results = await asyncio.gather(
        self.get_open_interest(),
        self.get_top_trader_positioning(),
        self.get_options_context(),
        self.get_taker_flow(),
        return_exceptions=True
    )
    return {
        "open_interest": results[0] if not isinstance(results[0], Exception) else {},
        "top_traders": results[1] if not isinstance(results[1], Exception) else {},
        "options": results[2] if not isinstance(results[2], Exception) else {},
        "taker_flow": results[3] if not isinstance(results[3], Exception) else {},
    }
}

```

■ notifier.py

Alert & Notification System

```
import logging
import httpx
import asyncio
from typing import Optional

logger = logging.getLogger(__name__)

class TelegramNotifier:

    def __init__(self, token: Optional[str], chat_id: Optional[str]):
        self.token = token
        self.chat_id = chat_id
        self.base_url = f"https://api.telegram.org/bot{token}/sendMessage" if token else None

    @property
    def is_active(self) -> bool:
        return bool(self.token and self.chat_id)

    async def send_message(self, text: str):
        if not self.is_active:
            return

        try:
            async with httpx.AsyncClient(timeout=10.0) as client:
                payload = {
                    "chat_id": self.chat_id,
                    "text": text,
                    "parse_mode": "HTML"
                }
                resp = await client.post(self.base_url, json=payload)
                resp.raise_for_status()
        except Exception as e:
            logger.error(f"[Telegram] Failed to send message: {e}")

    async def send_trade_alert(self, symbol: str, side: str, price: float, size: float, sl: float, tp: float, is_dry: bool = False):
        mode_str = "■ [DRY-RUN]" if is_dry else "■ [LIVE-TRADE]"
        emoji = "■" if side.lower() == "buy" else "■"

        msg = (
            f"<b>{mode_str} ENTRY</b>\n"
            f"■■■■■■■■■■■■■■■■■■■■\n"
            f"{emoji} <b>{side.upper()} {symbol}</b>\n"
            f"■ Price: <code>{price:.2f}</code>\n"
            f"■ Size: <code>{size:.6f}</code>\n"
            f"■ SL: <code>{sl:.2f}</code>\n"
            f"■ TP: <code>{tp:.2f}</code>\n"
            f"■■■■■■■■■■■■■■■■■■■■"
        )
        await self.send_message(msg)

    async def send_startup_alert(
        self,
        symbol: str,
        exchange: str,
        mode: str,
        leverage: int,
        interval: str,
        version: str = "v7",
    ):
        mode_emoji = "■" if mode == "DRY-RUN" else "■"
        msg = (
            f"<b>{mode_emoji} QUAD-DESK BOT ONLINE</b>\n"
            f"■■■■■■■■■■■■■■■■■■■■\n"
            f"■ <b>Exchange:</b> <code>{exchange.upper()}</code>\n"
            f"■ <b>Symbol:</b> <code>{symbol}</code>\n"
            f"■ <b>Leverage:</b> <code>{leverage}</code>\n"
            f"■ <b>Interval:</b> <code>{interval}</code>\n"
            f"■ <b>Mode:</b> <code>{mode}</code>\n"
            f"■ <b>Engine:</b> <code>7-Stage Hybrid {version}</code>\n"
            f"■■■■■■■■■■■■■■■■■■■■\n"
            f"<i>Bot is live and scanning markets.</i>"
        )
        await self.send_message(msg)

    async def send_error_alert(self, error_msg: str):
        msg = f"■■■ <b>BOT ERROR</b>\n■■■■■■■■■■■■■■■■■■■■\n<code>{error_msg}</code>"
        await self.send_message(msg)

    async def send_close_alert(self, symbol: str, side: str, price: float, type: str, pnl: float, is_dry: bool = False):
        mode_str = "■ [DRY-RUN]" if is_dry else "■ [LIVE-TRADE]"
        emoji = "■" if type == "SL" else "■"
        result = "PROFIT" if pnl >= 0 else "LOSS"
```

```
msg = (  
    f"<b>{mode_str} CLOSE</b>\n"  
    f"■■■■■■■■■■■■■■■■■■■■\n"  
    f"{emoji} <b>{type} HIT: {side.upper()} {symbol}</b>\n"  
    f"■ Exit Price: <code>{price:.2f}</code>\n"  
    f"■ {result}: <code>${pnl:.2f}</code>\n"  
    f"■■■■■■■■■■■■■■■■■■■■"  
)  
await self.send_message(msg)
```

Heartbeat & Monitoring

```
import os
import json
import logging
import asyncio
import time as _time
from typing import Optional, Any

logger = logging.getLogger(__name__)

# Module-level Firestore client (initialised once)
_db: Optional[Any] = None

_LOCAL_STATS_PATH = "/tmp/quad_bot_stats.json"
_LOCAL_TRADES_PATH = "/tmp/quad_bot_trades.jsonl"
_write_buffer: list = []
_consecutive_failures: int = 0
_use_local_fallback: bool = False
MAX_BUFFER_SIZE = 500

def _buffered_append(entry: dict) -> None:
    global _write_buffer
    if len(_write_buffer) >= MAX_BUFFER_SIZE:
        _write_buffer.pop(0)
    _write_buffer.append(entry)

def _normalize_pem(raw_key: str) -> str:
    if not raw_key: return ""

    # Step 1: Broad cleaning of escaping and quoting
    # Handle literal backslash-n, literal double-backslash-n, and true newlines
    key = str(raw_key).replace("\\\\n", "\n").replace("\\\\n", "\n")
    key = key.replace("\\r\\n", "\n").replace("\\r\\n", "\n").strip()
    key = key.strip("'").strip('"')

    # Step 2: Extract core body and headers
    lines = [l.strip() for l in key.splitlines() if l.strip()]
    if not lines: return ""

    header, footer = "", ""
    body_parts = []

    for l in lines:
        if "BEGIN" in l:
            header = l
        elif "END" in l:
            footer = l
        else:
            # [STRICTER] Base64 body should ONLY contain: A-Z, a-z, 0-9, +, /, and =
            # Any other characters (like backslashes or spaces) are artifacts.
            clean_part = "".join([c for c in l if c in "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/="])
            if clean_part:
                body_parts.append(clean_part)

    if not header or not footer:
        return key

    # Step 3: Flatten body and re-wrap strictly at 64 chars
    full_body = "".join(body_parts).rstrip("=")

    # 1.6 FIX: Validate key body length before padding.
    # Firebase RSA-2048 private key body = ~1700 base64 chars.
    # A body under 200 chars is almost certainly truncated by the secret manager.
    body_len = len(full_body)
    if body_len < 200:
        logger.error(
            f"[Heartbeat] Private key body is only {body_len} chars – "
            "likely truncated by Railway/secret manager character limit. "
            "Firebase RSA keys require ~1700 base64 chars. "
            "Paste the FULL JSON file content as the env var value."
        )
        return "" # Return empty; caller will skip Firebase init cleanly

    remainder = len(full_body) % 4
    if remainder == 2:
        full_body += "=="
    elif remainder == 3:
        full_body += "="
    elif remainder == 1:
        # remainder=1 is structurally invalid in base64 – key is definitely truncated.
        # Padding with '===' would produce a structurally valid but cryptographically
        # wrong key, hiding the real error. Fail loudly instead.
        logger.error(
            "[Heartbeat] Base64 body has remainder=1 – key is definitely truncated. "
            "Do NOT try to pad it. Store the COMPLETE JSON key as the env var."
        )
```

```

    )
    return ""

wrapped_body = [full_body[i:i+64] for i in range(0, len(full_body), 64)]

final_pem = "\n".join([header] + wrapped_body + [footer])
return final_pem

def _validate_pem(key: str) -> bool:

    lines = [l.strip() for l in key.splitlines() if l.strip()]
    if len(lines) < 3:
        return False
    has_header = any("BEGIN" in l for l in lines)
    has_footer = any("END" in l for l in lines)
    has_body = len(lines) >= 3 # at least header + 1 body line + footer
    return has_header and has_footer and has_body

def init_firebase() -> Optional[Any]:

    global _db
    cred_json = (
        os.environ.get("FIREBASE_ADMIN_CREDENTIALS", "").strip() or
        os.environ.get("FIREBASE_CREDENTIALS", "").strip()
    )
    if not cred_json:
        logger.warning(
            "[Heartbeat] FIREBASE_ADMIN_CREDENTIALS not set – "
            "bot↔dashboard live sync is disabled."
        )
        return None

    try:
        import firebase_admin
        from firebase_admin import credentials, firestore as fs

        cred_dict = json.loads(cred_json)

        # ■■ Fix PEM newlines (the root cause of 'InvalidPadding') ■■■■■■■■■■
        if "private_key" in cred_dict:
            raw = cred_dict["private_key"]
            normalized = _normalize_pem(raw)
            if not _validate_pem(normalized):
                logger.error(
                    "[Heartbeat] Firebase private_key PEM is malformed after normalization.\n"
                    " → Re-export the service account JSON from Firebase Console and paste\n"
                    "   the raw file contents (not the escaped string) into FIREBASE_ADMIN_CREDENTIALS."
                )
                return None
            cred_dict["private_key"] = normalized

        if not firebase_admin._apps:
            cred = credentials.Certificate(cred_dict)
            firebase_admin.initialize_app(cred)

        _db = fs.client()
        logger.info("[Heartbeat] Firebase connected ✓ – dashboard sync active.")
        return _db

    except json.JSONDecodeError:
        logger.error(
            "[Heartbeat] FIREBASE_ADMIN_CREDENTIALS is not valid JSON. "
            "Ensure the entire service account JSON is set as a single-line env var."
        )
    except Exception as e:
        logger.error(
            f"[Heartbeat] Firebase init failed: {e}\n"
            " → Fix: Re-export the service account key from Firebase Console → "
            "Project Settings → Service Accounts → Generate New Private Key.\n"
            " → Then set FIREBASE_ADMIN_CREDENTIALS to the full raw JSON content."
        )
    return None

import threading
import queue

class FirestoreLogHandler(logging.Handler):

    def __init__(self):
        super().__init__()
        self.log_queue = queue.Queue(maxsize=500)
        self.worker_thread = threading.Thread(target=self._log_worker, daemon=True)
        self.worker_thread.start()

    def _log_worker(self):
        while True:
            try:
                record = self.log_queue.get()
                if record is None:
                    break

```

[illegible]

```

        _buffered_append({"type": "status", **status_doc, "ts": _time.time()})
    else:
        try:
            await asyncio.to_thread(doc_ref.set, payload)
            _consecutive_failures = 0
            if backoff > 0:
                logger.info(f"[Heartbeat] Firebase recovered – backoff reset.")
                backoff = 0
            except Exception as e:
                _consecutive_failures += 1
                logger.warning(f"[Heartbeat] Firebase write error: {e}")
                if _consecutive_failures >= 3:
                    _use_local_fallback = True
                    logger.warning("[Heartbeat] 3 consecutive failures – switching to local JSON fallback.")
                    # FIXED: Alert operator that dashboard is dark
                    _tg_token = os.environ.get("TELEGRAM_BOT_TOKEN", "")
                    _tg_chat = os.environ.get("TELEGRAM_CHAT_ID", "")
                    if _tg_token and _tg_chat:
                        try:
                            import httpx as _httpx
                            asyncio.create_task(
                                _httpx.AsyncClient().post(
                                    f"https://api.telegram.org/bot{_tg_token}/sendMessage",
                                    json={
                                        "chat_id": _tg_chat,
                                        "text": (
                                            "■■■ Quad-Desk: Firebase is DARK\n"
                                            "3 consecutive write failures.\n"
                                            "Trade data → local /tmp fallback only.\n"
                                            "Check Firestore quota or credentials."
                                        )
                                    },
                                ),
                                timeout=5.0
                            )
                        except Exception:
                            pass
                    else:
                        backoff = min(backoff * 2 + 30, 300)
                        logger.info(f"[Heartbeat] Exponential backoff: {backoff}s")

# 5.2: Equity curve time-series point (every heartbeat cycle)
equity_doc = {
    "equity": stats.get("account_equity", 0.0),
    "sessionPnl": stats.get("session_pnl", 0.0),
    "dailyPnl": stats.get("daily_pnl", 0.0),
    "timestamp": fs.SERVER_TIMESTAMP,
}
if _use_local_fallback:
    # BUG-6 FIX: Use numeric timestamp for JSON serialization
    equity_doc["timestamp"] = _time.time()
    _buffered_append({"type": "equity", **equity_doc, "ts": _time.time()})
else:
    # P2-2 FIX: Day-bucketed documents instead of equity_ref.add() per minute.
    # Old: 1 new Firestore doc/minute = 86,400 docs after 60 days.
    # New: 1 doc/day with arrayUnion; merge=True creates doc if missing.
    try:
        from datetime import datetime
        _day_key = datetime.utcnow().strftime("%Y-%m-%d")
        _equity_point = {k: v for k, v in equity_doc.items() if k != "timestamp"}
        _equity_point["ts"] = _time.time()
        _equity_day_ref = _db.collection("equityCurve").document(_day_key)
        await asyncio.to_thread(
            _equity_day_ref.set,
            {"points": fs.ArrayUnion([_equity_point]), "date": _day_key},
            True,
        )
    except Exception as e:
        _consecutive_failures += 1
        logger.warning(f"[Heartbeat] Equity write error: {e}")

# Flush buffer to local JSON every BATCH_FLUSH_SECS
now = _time.time()
if _use_local_fallback and _write_buffer and (now - last_batch_flush) >= BATCH_FLUSH_SECS:
    _flush_local_buffer()
    last_batch_flush = now

except asyncio.CancelledError:
    break
except Exception as e:
    logger.warning(f"[Heartbeat] Firebase sync error (will retry): {e}")

try:
    await asyncio.sleep(WRITE_INTERVAL)
except asyncio.CancelledError:
    break

def _flush_local_buffer():
    global _write_buffer

```

```

if not _write_buffer:
    return
count = len(_write_buffer)
try:
    with open(_LOCAL_STATS_PATH, "a") as f:
        for entry in _write_buffer:
            f.write(json.dumps(entry) + "\n")
        _write_buffer.clear()
    logger.info(f"[Heartbeat] Flushed {count} entries to {_LOCAL_STATS_PATH}")
except Exception as e:
    logger.warning(f"[Heartbeat] Local JSON write failed: {e}")

async def write_offline(stats: dict) -> None:

    global _write_buffer
    if _use_local_fallback and _write_buffer:
        _flush_local_buffer()

    if _db is None:
        return

    try:
        from firebase_admin import firestore as fs
        doc_ref = _db.collection("botStatus").document("live")
        await asyncio.to_thread(doc_ref.set, {
            "isRunning": False,
            "lastHeartbeat": fs.SERVER_TIMESTAMP,
            "symbol": stats.get("symbol", "UNKNOWN"),
            "mode": stats.get("mode", "DRY-RUN"),
            "environment": stats.get("environment", "TESTNET"),
            "activePositions": 0,
            "totalTrades": stats.get("total_trades", 0),
            "lastSignal": "BOT_STOPPED",
            "exchange": stats.get("exchange", "coinbase").lower(),
            "lastUlis": "-",
        })
        logger.info("[Heartbeat] Offline status written to Firebase ✓")
    except Exception as e:
        logger.error(f"[Heartbeat] Offline write failed: {e}")

```


Backtesting Engine

[illegible]

```

log_ret = np.zeros(len(close))
log_ret[1:] = np.log(close[1:] / np.maximum(close[:-1], 1e-10))
skew = pd.Series(log_ret).rolling(period).skew().fillna(0).values
return skew

def calc_cvd_proxy(close, open_, volume, period=20, taker_buy_base=None):
    if taker_buy_base is not None:
        # Exact formula – mirrors the live frontend CVD calculation
        delta = (2.0 * taker_buy_base) - volume
    else:
        body = np.abs(close - open_)
        conv = np.clip(body / (body + 1e-10), 0.3, 1.0)
        dir_ = np.where(close >= open_, 1.0, -1.0)
        delta = dir_ * volume * conv
    cvd = pd.Series(delta).rolling(period).sum().fillna(0).values
    return cvd

# =====
# REAL DATA FETCHER – Binance Public REST API
# =====

def fetch_binance_candles(symbol: str, interval: str = "5m", years_back: int = 2) -> pd.DataFrame:
    try:
        import requests as _req
    except ImportError:
        print(" ■■■ `requests` not installed. Run: pip install requests")
        return pd.DataFrame()

    end_dt = datetime.now()
    start_dt = end_dt - timedelta(days=int(365 * years_back))
    start_ms = int(start_dt.timestamp() * 1000)
    end_ms = int(end_dt.timestamp() * 1000)

    print(f" Fetching {symbol} ({interval}) from Binance API "
          f"[{start_dt.date()} → {end_dt.date()}]...")

    all_klines: list = []
    current_ms = start_ms

    while current_ms < end_ms:
        try:
            resp = _req.get(
                "https://api.binance.com/api/v3/klines",
                params={"symbol": symbol,
                       "interval": interval,
                       "startTime": current_ms,
                       "endTime": end_ms,
                       "limit": 1000},
                timeout=20,
            )
            resp.raise_for_status()
            batch = resp.json()
        except Exception as exc:
            print(f" ■■■ Binance API error: {exc}. Falling back to CSV / synthetic data.")
            return pd.DataFrame()

        if not batch:
            break
        all_klines.extend(batch)
        current_ms = int(batch[-1][0]) + 1
        if len(batch) < 1000:
            break

    if not all_klines:
        return pd.DataFrame()

    cols = [
        'time', 'open', 'high', 'low', 'close', 'volume',
        'close_time', 'quote_vol', 'trades',
        'taker_buy_base', 'taker_buy_quote', 'ignore',
    ]
    df = pd.DataFrame(all_klines, columns=cols)
    df['time'] = pd.to_datetime(df['time'].astype(int), unit='ms')
    df.set_index('time', inplace=True)
    for col in ('open', 'high', 'low', 'close', 'volume', 'taker_buy_base'):
        df[col] = df[col].astype(float)

    print(f" ✓ {len(df):,} real bars loaded "
          f"[{df.index[0].date()} → {df.index[-1].date()}]")
    return df

# =====
# STRATEGY & BAYESIAN FUSION
# =====

def backtest_hybrid(df, config, symbol):
    close = df['close'].values
    high = df['high'].values

```

[illegible]

```

# How much coin to buy so that sl_distance == risk_usd
size_coin = risk_usd / max(sl_distance, 1e-9)
notional_value = size_coin * t['entry']

# Raw directional PnL based on nominal cost vs exit
price_diff = (eff_exit - eff_entry) if is_long else (eff_entry - eff_exit)
raw_pnl = size_coin * price_diff

# Commission is paid on total notional volume traded (Entry + Exit)
exit_notional = size_coin * eff_exit
commission_cost = (notional_value + exit_notional) * comm

pnl = raw_pnl - commission_cost
r_mult = pnl / max(risk_usd, 1e-9)

exit_px = raw_exit # keep original for logging
balance += pnl
if pnl < 0:
    daily_losses += 1
    if daily_losses >= int(config['max_daily_loss_pct']):
        daily_halt = True

trades.append({
    "entry_time": t['entry_time'],
    "exit_time": ts[i],
    "dir": "BUY" if t['dir'] == 1 else "SELL",
    "entry": t['entry'],
    "exit": exit_px,
    "pnl": pnl,
    "r_mult": r_mult,
    "balance": balance,
    "regime": t['regime']
})
won = (pnl > 0)
trade_regime = t['regime']
if won:
    backtest_alpha += 1.0
    backtest_regime_alpha[trade_regime] = backtest_regime_alpha.get(trade_regime, 1.0) + 1.0
else:
    backtest_beta += 1.0
    backtest_regime_beta[trade_regime] = backtest_regime_beta.get(trade_regime, 1.0) + 1.0
    backtest_regime_count[trade_regime] = backtest_regime_count.get(trade_regime, 0) + 1
else:
    still_open.append(t)
open_trades = still_open

if daily_halt or len(open_trades) > 0:
    continue

# 2. Market Regime
regime = "NEUTRAL"
# Proximate to recent swing high/low (simulated walls)
near_wall = False
nav_h = sw_high[i]
nav_l = sw_low[i]

if not np.isnan(nav_h) and not np.isnan(nav_l):
    if abs(px - nav_h) / px <= 0.001 or abs(px - nav_l) / px <= 0.001:
        regime = "LIQUIDITY"
        near_wall = True

if not near_wall:
    # Mirror live bot Stage 2: TREND requires high ATR AND a volume surge
    # (proxy for the SCREAMING tape condition checked against Binance trade feed)
    sma_vol = vol_sma_60[i] if i < len(vol_sma_60) else 0.0
    vol_surge = sma_vol > 0 and vol[i] > sma_vol * 3.0
    if atr_pct[i] > config['trend_atr_threshold'] and vol_surge:
        regime = "TREND"
    elif abs(zscore[i]) < config['range_z_threshold']:
        regime = "RANGE"

# 3. Liquidity Sweep Detection
sweep = None
if not np.isnan(nav_h) and not np.isnan(nav_l):
    prev_h, prev_l = high[i-1], low[i-1]
    prev_c = close[i-1]
    if prev_h > nav_h and px < nav_h:
        sweep = "ABOVE_HIGHS"
    elif prev_l < nav_l and px > nav_l:
        sweep = "BELOW_LOWS"

# 4. Synthese Pre-Bayes matching live `quant_engine.py`
curr_rsi = rsi[i]
curr_z = zscore[i]
curr_skew = skew[i]
curr_ofi = ofi_normalised[i] # tanh scale (-1, +1), replaces raw proxy * 10.0

L_rsi = 1.8 if curr_rsi > 60 else 0.55 if curr_rsi < 40 else 1.0
if curr_z < -1.22 and curr_ofi > 0.15:
    L_flow = 2.0

```

```

elif curr_z > 1.22 and curr_ofi < -0.15:
    L_flow = 0.5
else:
    L_z = 1.3 if curr_z < -1.22 else 0.76 if curr_z > 1.22 else 1.0
    L_o = 1.2 if curr_ofi > 0.30 else 0.83 if curr_ofi < -0.30 else 1.0
    L_flow = L_z * L_o

L_skew = 1.2 if curr_skew > 0.3 else 0.83 if curr_skew < -0.3 else 1.0
p_prior = backtest_alpha / (backtest_alpha + backtest_beta)
prior_odds = p_prior / (1.0 - p_prior)
bull_odds = prior_odds * L_rsi * L_flow * L_skew
bayes = bull_odds / (bull_odds + 1.0)

# 5. Strategy Layer
raw_direction = None
strategy = ""

if sweep:
    strategy = "SWEEP"
    if sweep == "ABOVE_HIGHS" and (curr_ofi < -0.15 or cvd[i] < 0):
        raw_direction = "SELL"
    elif sweep == "BELOW_LOWS" and (curr_ofi > 0.15 or cvd[i] > 0):
        raw_direction = "BUY"

elif regime == "TREND":
    strategy = "TREND"
    score = 0.0

    if bayes > 0.65: score += 2.0
    elif bayes > 0.55: score += 1.0
    elif bayes < 0.35: score -= 2.0
    elif bayes < 0.45: score -= 1.0

    if curr_ofi > 0.30: score += 1.5
    elif curr_ofi > 0.15: score += 0.75
    elif curr_ofi < -0.30: score -= 1.5
    elif curr_ofi < -0.15: score -= 0.75

    if cvd[i] > 0: score += 1.0
    elif cvd[i] < 0: score -= 1.0

    if curr_ofi > 0: score += 1.0
    elif curr_ofi < 0: score -= 1.0

    if score >= 1.5: raw_direction = "BUY"
    elif score <= -1.5: raw_direction = "SELL"

elif regime == "RANGE":
    strategy = "MEAN_REVERSION"
    atr_rank = atr_pct_rank[i]
    if atr_rank < 0.5:
        rsi_long_gate = 55.0
        rsi_short_gate = 45.0
    else:
        rsi_long_gate = 42.0
        rsi_short_gate = 58.0
    if curr_z >= 1.06 and curr_rsi > rsi_short_gate: raw_direction = "SELL"
    elif curr_z <= -1.06 and curr_rsi < rsi_long_gate: raw_direction = "BUY"

if not raw_direction: continue

# 6. Bayesian Fusion + ULIS proxy
odds = 1.0
if curr_ofi > 0.30: odds *= 2.0
elif curr_ofi > 0.15: odds *= 1.4
elif curr_ofi < -0.30: odds *= 0.5
elif curr_ofi < -0.15: odds *= 0.7

if cvd[i] > 0: odds *= 1.25
elif cvd[i] < 0: odds *= 0.8

if curr_skew > 0.3: odds *= 1.12
elif curr_skew < -0.3: odds *= 0.88

if curr_rsi > 60: odds *= 1.15
elif curr_rsi < 40: odds *= 0.85

p_bull = odds / (odds + 1.0)
conf = p_bull if raw_direction == "BUY" else (1.0 - p_bull)

# CVD directional gate: penalise trades where CVD contradicts direction.
# Mirrors Stage 4.5 CVD divergence logic in the live bot.
cvd_confirms_long = (raw_direction == "BUY" and cvd[i] > 0)
cvd_confirms_short = (raw_direction == "SELL" and cvd[i] < 0)
if not (cvd_confirms_long or cvd_confirms_short):
    conf -= 0.04 # Mild penalty for CVD-contradicting direction

if (raw_direction == "BUY" and curr_ofi < 0 and curr_z < 0) or \
    (raw_direction == "SELL" and curr_ofi > 0 and curr_z > 0):
    conf -= 0.15 # ULIS cascade veto penalty simulation

```

[illegible]

```

np.random.seed(42)
end_date = datetime.now()
try:
    start_date = end_date.replace(year=end_date.year - years_back)
except ValueError:
    start_date = end_date.replace(year=end_date.year - years_back, day=end_date.day - 1)

periods = int((end_date - start_date).total_seconds() / 300) # 5-min bars
dates = pd.date_range(end=end_date, periods=periods, freq="5min")

returns = np.random.normal(0, 0.001, periods) * np.abs(np.random.normal(1, 0.2, periods))
close = 30_000.0 * np.exp(np.cumsum(returns)) # BTC-realistic starting price
high = close * (1 + np.abs(np.random.normal(0, 0.0005, periods)))
low = close * (1 - np.abs(np.random.normal(0, 0.0005, periods)))
open_ = np.roll(close, 1); open_[0] = close[0]
vol = np.random.lognormal(mean=10, sigma=1, size=periods)

return pd.DataFrame(
    {'open': open_, 'high': high, 'low': low, 'close': close, 'volume': vol},
    index=dates,
)

def run():
    print(f"\n{Fore.CYAN}{Style.BRIGHT}{█*62}")
    print(f" Macro Hybrid Bot – Vectorised Backtester")
    print(f" 10-Year Test Engine")
    print(f"{█*62}{Style.RESET_ALL}\n")

    t_start = time.time()

    for symbol in CONFIG['symbols']:
        print(f" {Fore.CYAN}█ {symbol}{Style.RESET_ALL}")
        # Try to load local CSV if it exists, otherwise use synthetic
        csv_path = f"data/{symbol}_M5.csv"
        if os.path.exists(csv_path):
            print(f" Loading {csv_path}...")
            df = pd.read_csv(csv_path, index_col=0, parse_dates=True)
            df.sort_index(inplace=True)
        else:
            df = load_data(symbol)

        print(f" Bars: {len(df):,} ({df.index[0].date()} -> {df.index[-1].date()})")

        trades, final_bal = backtest_hybrid(df, CONFIG, symbol)

        # Report
        df_t = pd.DataFrame(trades)
        if len(df_t) == 0:
            print(f" {Fore.RED}No trades executed.{Style.RESET_ALL}")
            continue

        wins = (df_t['pnl'] > 0).sum()
        total = len(df_t)
        wr = wins / total * 100
        net = df_t['pnl'].sum()

        print(f"\n Results for {symbol}:")
        print(f" Initial Balance: ${CONFIG['initial_balance']:,.2f}")
        color = Fore.GREEN if net > 0 else Fore.RED
        print(f" Final Balance: {color}${final_bal:,.2f}{Style.RESET_ALL}")
        print(f" Net Profit: {color}${net:,.2f}{Style.RESET_ALL}")
        print(f" Total Trades: {total}")
        print(f" Win Rate: {wr:.1f}%")

        # Breakdown by regime
        print("\n Performance by Regime:")
        by_regime = df_t.groupby('regime').agg(
            trades=('pnl', 'count'),
            win_rate=('pnl', lambda x: (x>0).mean() * 100),
            pnl=('pnl', 'sum')
        )
        print(by_regime.to_string())
        print(f"\n{Fore.CYAN}{Style.BRIGHT}{█*62}{Style.RESET_ALL}\n")

    print(f" Total time: {time.time() - t_start:.2f}s")

if __name__ == "__main__":
    run()

```